

Prompting Frameworks for Large Language Models: A Survey

XIAOXIA LIU, Zhejiang University, Hangzhou, China

JINGYI WANG, Zhejiang University, Hangzhou, China

XIAOHAN YUAN, Zhejiang University, Hangzhou, China

JUN SUN, Singapore Management University School of Information Systems, Singapore, Singapore

GUOLIANG DONG, Singapore Management University, Singapore, Singapore

PENG DI, Ant Group CO Ltd, Hangzhou, China

WENHAI WANG, Zhejiang University, Hangzhou, China

DONGXIA WANG, Zhejiang University, Hangzhou, China

Since the launch of ChatGPT, a powerful AI Chatbot developed by OpenAI, large language models (LLMs) have made significant advancements in both academia and industry, bringing about a fundamental engineering paradigm shift in many areas. While LLMs are powerful, it is also crucial to best use their power where “prompt” plays a core role. However, the booming LLMs themselves, including excellent APIs like ChatGPT, have several inherent limitations: (1) temporal lag of training data, and (2) the lack of physical capabilities to perform external actions. Recently, we have observed the trend of utilizing prompt-based tools to better utilize the power of LLMs for downstream tasks, but a lack of systematic literature and standardized terminology, partly due to the rapid evolution of this field. Therefore, in this work, we survey related prompting tools and promote the concept of the “Prompting Framework” (PF), i.e. the framework for managing, simplifying, and facilitating interaction with LLMs. We define the lifecycle of the PF as a hierarchical structure, from bottom to top, namely: Data Level, Base Level, Execute Level, and Service Level. We also systematically depict the overall landscape of the emerging PF field and discuss potential future research and challenges. To continuously track the developments in this area, we maintain a repository at <https://github.com/lxx0628/Prompting-Framework-Survey>, which can be a useful resource sharing platform for both academic and industry in this field.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; • **Software and its engineering** → **Development frameworks and environments**;

Additional Key Words and Phrases: Large language models, prompting

We thank the anonymous reviewers for their constructive comments to improve our article. This research is supported by the Ministry of Education, Singapore under its Academic Research Fund Tier 3 (Award ID: MOET32020-0004) and the Key R&D Program of Zhejiang under Grant No. 2025C01083. This work was supported by Ant Group.

Authors' Contact Information: Xiaoxia Liu, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: liuxiaoxia@zju.edu.cn; Jingyi Wang, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: wangjiyee@zju.edu.cn; Xiaohan Yuan, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: xiaohanyuan@zju.edu.cn; Jun Sun, Singapore Management University School of Information Systems, Singapore, Singapore; e-mail: junsun@smu.edu.sg; Guoliang Dong, Singapore Management University, Singapore, Singapore, Singapore; e-mail: gldong@smu.edu.sg; Peng Di, Ant Group CO Ltd, Hangzhou, China; e-mail: pengdi@cse.unsw.edu.au; Wenhai Wang, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: zdzzlab@zju.edu.cn; Dongxia Wang (corresponding author), Zhejiang University, Hangzhou, China; e-mail: dxwang@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 0360-0300/2026/04-ART255

<https://doi.org/10.1145/3789253>

ACM Reference Format:

Xiaoxia Liu, Jingyi Wang, Xiaohan Yuan, Jun Sun, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. 2026. Prompting Frameworks for Large Language Models: A Survey. *ACM Comput. Surv.* 58, 10, Article 255 (April 2026), 38 pages. <https://doi.org/10.1145/3789253>

1 Introduction

Since the release of ChatGPT,¹ which attracted widespread social attention, research on **large language models (LLMs)** has been in full swing in both academia and industry, resulting in a number of amazing products such as PaLM [27], GPT-4 [96], and LLaMA [124, 125]. These LLMs have been shown to exhibit remarkable capabilities in approaching or even exceeding human-level performance in dialogue, text translation, and sentiment analysis [2, 11, 25, 61], and so on, potentially bringing in fundamental changes of many fields [18, 30, 38, 68, 74, 89, 139, 154]. The development of **language models (LMs)** to the current flourishing state has undergone a series of evolutionary processes: *fully supervised learning* → *deep learning for NLP* → “Pre-train, Fine-tune” → “Pre-train, Prompt, Predict” [67, 151]. Initially, LMs applied a fully supervised learning paradigm, where task-specific models were trained solely on the target task dataset, heavily relying on feature engineering [58, 93, 114]. Subsequently, with the rise of deep learning, neural networks for NLP emerged, enabling the integration of feature learning and model training, i.e., a network architecture designed to automatically learn data features [7, 8, 29, 83]. Later, as the demand for LMs increased and to accommodate the growing number of NLP tasks, the “Pre-train, Fine-tune” paradigm was introduced. In this paradigm, a model with a fixed architecture undergoes pre-training to predict the probability of observed text data. Additional parameters are then introduced, and the model is fine-tuned using task-specific objective functions to adapt the pre-trained LM to various downstream tasks [62, 116, 127, 143]. Then came the era of LLMs, where the trend shifted toward downstream tasks actively adapting to pre-trained models. The paradigm of “Pre-train, Prompt, Predict” became mainstream and prompts successfully empowering the LLMs to effortlessly tackle a wide range of complex and diverse tasks. By providing a suitable set of prompts, a single LM trained entirely on context-based predictions can be employed to address various tasks [13, 112]. Therefore, the quality and appropriateness of prompts are increasingly playing a crucial role in task resolution [55, 136, 153]. Both the academic and industrial communities have shown growing attention and interest in research related to prompts.

Numerous studies have demonstrated the necessity of employing appropriate methods to unleash the potential of LLMs [132, 136, 145, 153]. In March 2023, OpenAI officially unveiled a significant innovation known as ChatGPT plugins, which enable ChatGPT to utilize external tools, reflecting a clear response to the growing demand for enhancing LLMs’ interaction capabilities with the external world. When analogized to humans, LLMs can be regarded as the intelligent system’s brain, responsible for perceiving instructions and generating and controlling a series of actions. Therefore, by combining their inherent knowledge and capabilities with external tools such as search engines, computational utilities, visual models, and more, LLMs can perform a wide array of real-world tasks, including real-time data retrieval, browser-based information retrieval, database access, precise mathematical calculations, complex language generation, and image analysis, thus showcasing their potential across diverse domains like education, healthcare, social media, finance, and natural sciences [73, 79, 91, 110]. Consequently, *the development of tools that facilitate the optimization and streamlining of the interaction process becomes crucial*. In this article, we collectively refer to these forward-looking tools as a proposed novel concept: “**Prompting Framework**” (PF).

¹<https://openai.com/blog/chatgpt/>

In general, the PF is the upper layer that enables LLMs to interact with the external world. A PF *manages, simplifies, and facilitates such interactions*, helping LLMs overcome fundamental challenges like data lag or “brain in a vet”. Moreover, PFs also serve as the basic infrastructure of recently emerging autonomous agents based on LLMs, such as AutoGPT [119], HuggingGPT [138], and MetaGPT [49].

Since the release of the open-source project LangChain [21] by Harrison Chase in October 2022, it has garnered attention from over 60,000 supporters on GitHub and stands as one of the most popular PFs to date. LangChain is a framework for building applications with LLMs through composability. Besides LangChain, our investigation encompasses various kinds of state-of-the-art PFs, including (1) Semantic Kernel [128], LlamaIndex [66], and OpenDAN [97], which can be arguably considered as the operating systems for LLMs, as well as (2) output restrictors for LLMs such as Guidance [80], TypeChat [81], NeMo-Guardrails [92], and (3) language for interacting with LLMs, such as LMQL [10], gpt-jargon [14], SudoLang [39]. When referring to PFs, a notable challenge arises due to the rapid pace of development in the domain, making it difficult to track and stay informed about the multitude of methods dispersed across GitHub, preprint papers, X (formerly known as Twitter), and top conferences/journals. Furthermore, the abundance of PF approaches with varying focuses makes it challenging to systematically categorize and compare them, hindering the selection of the most suitable product for specific needs. Therefore, there is currently a lack of but an urgent need for systematic literature and standardized terminology for introducing and comparing these tools that are essential for better using LLMs’ capabilities.

In this survey, we introduce the concept of ‘Prompting Framework’, and provide a comprehensive and systematic survey of existing PFs. We present formal definitions, categorization, comparative analysis, and evaluation criteria for them, assess their applicability and limitations, and provide practical recommendations for their effective utilization for real-world LLM-enabled tasks. We also examined several works and toolkits that, although outside the scope of the PF, can support and enhance interactions with LLMs. These resources contribute to improving the effectiveness of the PF, optimizing its performance, and expanding its applicability. We also present recommendations for future research. In a nutshell, we make the following main contributions:

- We introduce the concept of PFs that garnered attention in both academia and industry, and provided systematic and standardized definitions and terminology.
- We categorize the existing PFs into three classes, conduct a comprehensive comparison of their strengths and limitations across various dimensions, and provide practical recommendations. Based on the research findings, we present the future directions of the PF and extensively explore its potential development and challenges in more domains.
- We conduct extensive research beyond the scope of PFs, including works and toolkits that can synergistically support and enhance interactions with LLMs in conjunction with PFs.

The rest of the article is structured as follows and the structure is in Figure 1. Section 2 presents background knowledge of the PF, including the characteristics of LLMs and the necessity of the PF. Section 3 describes the investigation, including the methodologies and results. Section 4 reviews prompt-related key conceptions, including prompt, prompting, prompt engineering, and prompt learning. Section 5 provides the systematic definitions and taxonomy of PFs. Section 6 presents the comparison and challenges across various dimensions of various PFs. Section 7 reviews prompt-related works and toolkits outside the scope of the PF, but can synergistically support and enhance interactions with LLMs in conjunction with PFs. Section 8 presents the future directions of the PF and the potential developments and challenges in more domains.

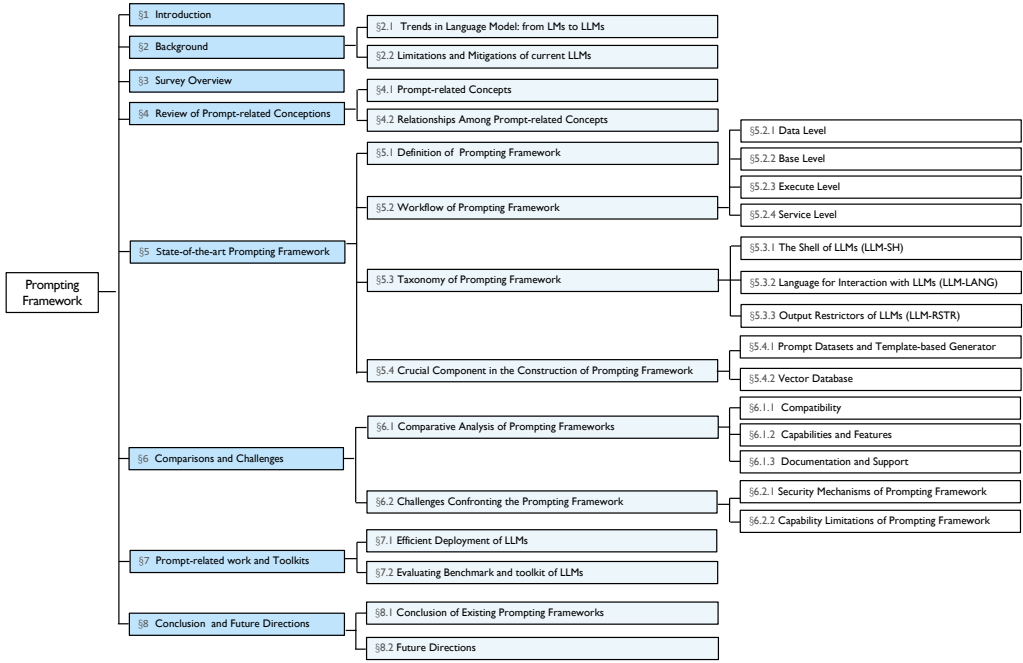


Fig. 1. The structure of this article.

2 Background

In this section, we present the background of the PF, including the reasons behind its emergence and the pertinent terminologies. We aim to address the following aspects: (1) elucidating the concept of LLMs by tracing their development history, and (2) explicating the current capability limitations of LLMs to underscore the necessity for the PF.

2.1 Trends in Language Model: From LMs to LLMs

Early LMs were predictive models based on Markov assumptions using statistical learning methods, also known as **Statistical language models (SLM)** [53, 56, 93, 122]. However, due to the limitations imposed by the fully supervised learning approach, the curse of dimensionality was inevitably a challenge. With the rise of deep learning, researchers turned to neural networks to enhance LM’s capabilities, leading to the emergence of **Neural Language Models (NLMs)** [82, 84]. NLMs aim to establish a universal neural network framework for various **natural language processing (NLP)** tasks. Subsequently, the introduction and popularity of the Transformer architecture and self-attention mechanism [127] gave rise to a series of task-agnostic pre-trained models, such as BERT and GPT, called **Pre-trained language models (PLM)**, which promoted the emergence of the “pre-train, fine-tune” paradigm [67]. PLMs have exhibited remarkable performance improvements across a wide range of NLP tasks [33, 62, 70, 111].

To further explore the performance of LMs, researchers have continuously increased the scale of model parameters, the trend has shifted toward downstream tasks actively adapting to pre-trained models. The paradigm of “Pre-train, Prompt, Predict” became mainstream [67]. Prompts are an important medium for interaction with LMs, usually in text form. In this process, the augmented models not only exhibit better performance on various NLP tasks but also demonstrate remarkable “emergent abilities” [135], which were previously unseen in smaller PLMs with similar architectures.

For instance, ChatGPT can mimic human language style and logical reasoning and also demonstrates outstanding contextual comprehension, which was absent in previous models like GPT-2. Based on this new capability distinction, researchers refer to these emerging PLMs with hundreds of billions of parameters as LLMs [50, 55, 136], such as ChatGPT, and GPT-4 [96]. With this advancement, LMs have completed the leap from LMs to LLMs and inspiring new prospects for **artificial general intelligence (AGI)**.

2.2 LLMs still “Brains in a Vat”: Limitations and Mitigations

Analogous to humans, LLMs can be perceived as the brains of artificial intelligence systems, responsible for perceiving instructional information and generating and controlling actions. Although there has been evidence of “Emergent Abilities” [85, 133, 135, 141] in LLMs, which refers to the abilities that emerge in large-scale models but have not been observed in smaller models and can be primarily categorized into four aspects: in-context learning [36, 86, 131, 137], reasoning for complex content [63, 136, 153], instruction following [28, 99, 102, 132], and creative capacity [16, 37, 149, 150].

However, the capacity limitations of LLMs cannot be ignored. Firstly, LLMs suffer from temporal lag in their training data, such as ChatGPT’s latest training data being limited to September 2021 (at the time of article writing). LLMs are unable to access real-time information and trends, and they may struggle to accurately comprehend specific terminology or domain-specific knowledge, occasionally leading to incomplete or erroneous responses, even illusions [17, 20, 25, 32, 40, 64, 129]. Additionally, LLMs are bound by strict limitations on the number of tokens they can process during interactions, severely restricting the amount of contextual information they can take in and process [9, 15, 142, 148]. Secondly, LLMs are incapable of direct interaction with external expert models, such as utilizing search engines, querying databases, and invoking external tools, or APIs, which limits their usability [79, 110, 151]. Furthermore, the majority of LLMs are offered as paid APIs, potentially imposing financial burdens on individuals, organizations, and projects with limited resources when dealing with large-scale or frequent requests [23, 98].

Based on the above challenges, it is thus desirable to overcome these barriers by bridging the gap between LLMs and external applications. The adoption of PFs, such as Langchain and semantic kernel, becomes imperative [22, 89]. These frameworks not only enable LLMs to stay constantly exposed to emerging information but also enable the processing of long texts and documents, and facilitate seamless integration with external applications.

3 Survey Overview

This section outlines our survey process. The rapid development of LLMs and related technologies has created a dispersed research landscape, with many contributions yet to be formally published. These developments often appear on platforms like arXiv, GitHub, or within online communities (e.g., X, Discord), lacking formal documentation. Additionally, there is a scarcity of comprehensive review literature, leaving a gap in established terminology and definitions.

Our exploration begins with LangChain, one of the most influential frameworks in this area. We analyze its official description, which emphasizes “building applications with LLMs through composability.” This phase establishes key concepts such as “frameworks,” “tools,” “Agents,” “Large Models,” “prompts,” and “toolkits,” which guide our review and help us include a wide range of relevant resources. To ensure thoroughness, we conduct extensive keyword searches across repositories like GitHub, scholarly databases (e.g., arXiv), and relevant AI/NLP journals and conferences. This approach yields 62 open-source GitHub projects and numerous academic papers, forming the foundation of our research on PFs.

We then perform a detailed evaluation of these 62 works, starting with a review of their technical documentation. We assess the conceptual foundations, implementations, and key code segments,

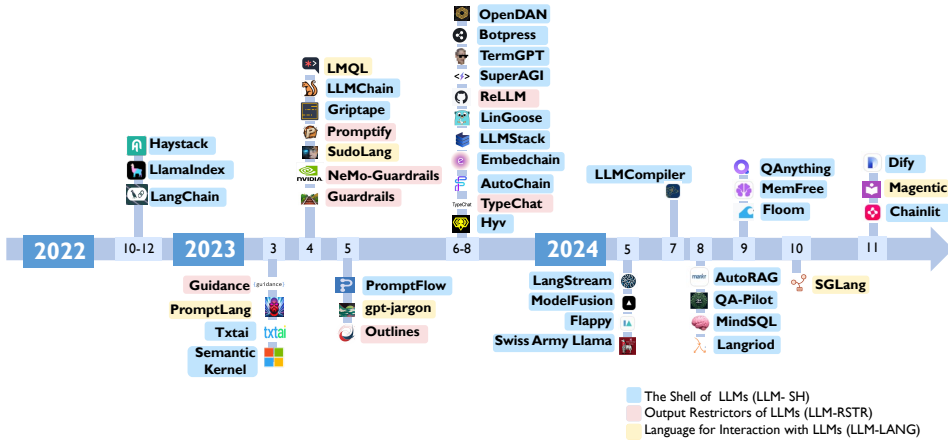


Fig. 2. The timeline of representative prompting frameworks.

configuring and testing the tools to evaluate their performance and applicability. Our testing process includes unit testing of individual modules, integration testing of assembled modules, and comprehensive system testing to validate claimed capabilities and user experience. From this rigorous analysis, we identify the strengths and weaknesses of each work. Ultimately, we select around 30 projects that meet the conceptual requirements and stand out in the field, ensuring a representative sample of the evolving landscape of PFs. Figure 2 is a chronological overview of the representative PF in recent years.

4 Review of Prompt-Related Concepts

The rise of generative AI has made prompt-driven interaction central to NLP. To leverage this paradigm effectively, it is crucial to understand key concepts such as Prompt, Prompting, Prompt Engineering, and Prompt Learning. This section reviews these concepts and their interrelations. The relationships are illustrated in Figure 3.

4.1 Prompt-Related Concepts

In this section, we clarify the key concepts related to “prompt”.

Prompt. A prompt is a structured input to a LM, typically in natural language, used to convey user intent. It provides task-specific instructions or queries, guiding the model to generate relevant outputs. Prompts can vary in complexity, from simple commands to detailed, context-aware templates designed for clarity and task alignment.

Prompting. Prompting refers to the process of interacting with a LM through prompts. It involves preparing prompts (e.g., refining syntax, adding context), delivering them to the model, and retrieving the outputs. This process serves as the interface between the user and the model, ensuring effective input-output interaction within the model’s computational architecture.

Prompting Engineering. Prompt Engineering is the systematic discipline of designing, refining, and optimizing prompts to maximize their effectiveness in eliciting desired responses from a LM. This process involves advanced techniques such as dynamic example selection, context augmentation, template generation, and optimization algorithms. By leveraging tools such as context management and memory modules, it enhances semantic accuracy, reduces ambiguity, and aligns prompts with task objectives.

Prompt Learning. Prompt Learning describes the iterative adaptive process through which a LM incrementally refines its internal representations and enhances its generative and interpretive capabilities in response to prompts. This involves training, fine-tuning, and updating the model’s parameters. Through these adaptations, the model enhances its ability to generalize across tasks and better align with user goals.

Prompting Framework. The PF, defined in the next section (Section 5), is a modular infrastructure that manages the entire lifecycle of prompt-based interactions. It includes components for input/output management, model management, task orchestration, data processing, and context handling. This system ensures the efficient execution of prompt-driven workflows, supporting optimization through Prompt Engineering, improvement via Prompt Learning, and seamless user interaction through Prompting.

4.2 Relationships Among Prompt-Related Concepts

As shown in Figure 3, the concepts of Prompt, Prompting, Prompt Engineering, and Prompt Learning are interrelated and mutually supportive. A Prompt is the initial input, defining user intent and task requirements. Prompting refers to its execution, ensuring the prompt is effectively interpreted by the model. Prompt Engineering improves prompt quality and task relevance through systematic design, enhancing the interaction and output of Prompting. Prompt Learning uses the Prompt to refine the model’s parameters, improving its generative capabilities. Feedback loops between these components enable continuous optimization: insights from Prompting inform Prompt Engineering, and results from Prompt Learning guide prompt design. Together, they create a cohesive system for optimizing LM interactions within the PF.

Consider a model tasked with identifying buffer overflow vulnerabilities in C code. The user provides a general prompt, e.g., “Detect vulnerabilities in the following C code” (prompt). The model processes the code, analyzing syntax, memory handling, and buffer operations to identify potential overflows (prompting). To improve accuracy, the engineer refines the prompt, e.g., “Detect buffer overflow vulnerabilities in C code that takes user input without bounds checking” (prompt engineering). As the model detects vulnerabilities and receives feedback (e.g., validation from experts or attack scenarios), it improves its ability to spot similar issues in future prompts (prompt learning). Over time, this feedback loop enhances the model’s detection of subtle vulnerabilities, optimizing results as it encounters more varied code and patterns.

5 State-of-the-Art Prompting Framework

In this section, we provide a systematic and comprehensive definition and description of the PF, which is essential for overcoming the application barriers of LLMs and enhancing their role as critical tools in real-world scenarios. Additionally, we classify state-of-the-art PFs, offering a systematic review of various approaches.

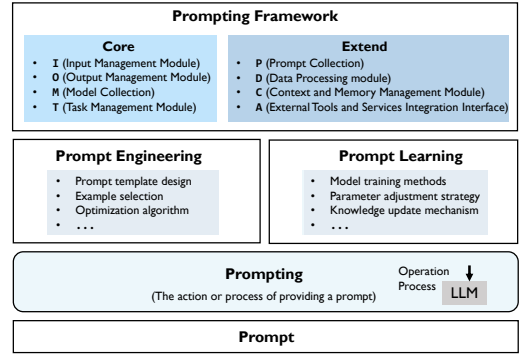


Fig. 3. The relationship between Prompt, Prompting, Prompt Engineering, Prompt Learning, and Prompting Framework.

5.1 Definition of Prompting Framework

The PF is a specialized system for interacting with LLMs, distinguishing itself from traditional frameworks while retaining core principles like modularity, abstraction, extensibility, and standardization. Modularity involves dividing the framework into independent modules for easier management and reuse. Abstraction offers simplified interfaces to hide complex implementation details. Extensibility allows users to customize the framework's functionality. Standardization ensures consistency in development for improved maintainability and readability.

The framework overcomes LLM limitations by enabling dynamic prompt generation and optimization, adapting to different tasks and scenarios. Unlike traditional fixed workflows, it enhances flexibility and performance through real-time adjustments. Its hierarchical design facilitates efficient collaboration across layers for data processing, task execution, and management, improving information flow and coherence. The framework also integrates with external tools, such as databases and search engines, based on task requirements, ensuring robustness in dynamic environments. Furthermore, it simplifies user interactions with template-based designs and provides efficient tools for developers, enhancing productivity. In summary, the PF extends LLM functionality, optimizes interactions, and overcomes the constraints of traditional frameworks, creating a new paradigm for LLM applications. It combines the strengths of traditional software frameworks with solutions tailored to LLM-specific needs. Based on this, we define the PF as follows.

Definition 1. Prompting Framework (PF). The Prompting Framework is a systematic architecture designed for managing, simplifying, and facilitating interactions with LLMs. The framework consists of two primary components: Core PF_{Core} and Extend PF_{Extend} , where Core represents the essential functionality required for the basic operation of the framework and Extend provides additional features to enhance its capabilities. Specifically, we define the framework PF as:

$$PF = (PF_{Core}, PF_{Extend})$$

with the following properties:

- $PF_{Core} = (M, T, I, O)$, i.e., the Core component must include all four modules: M (Model Collection), T (Task Management Module), I (Input Management Module), and O (Output Management Module).
- $PF_{Core} \subseteq PF$, i.e., a PF must include at least the complete Core component PF_{Core} .

Core Component. The Core component is the essential part of the PF, and any implementation that satisfies the Core requirements is considered a valid instance of PF. The Core is a quadruple, which consists of four key modules, which are formally defined as

$$PF_{Core} = (M, T, I, O)$$

where each module is defined as follows:

- **Model Collection (M):** This module defines the set of LMs available for interaction within the framework.
- **Task Management Module (T):** This module manages and organizes tasks that the system aims to complete through interactions with the LMs, and is also responsible for coordinating interactions between modules and controlling the flow of information.
- **Input Management Module (I):** This module is responsible for preprocessing and structuring the input data to ensure it is correctly formatted for efficient processing by the LMs.

- **Output Management Module (O):** This module handles the processing, formatting, and presentation of the outputs generated by the LMs.

Extend Component. The Extend component is optional and includes additional modules that extend the framework’s functionality. We define the **Extend** component as

$$PF_{\text{Extend}} = (C, A, D, P)$$

where each module is defined as follows:

- **Context and Memory Management Module (C):** This module manages context and memory for interactions with the LMs, allowing the system to track historical information and maintain coherent conversations.
- **External Tools and Services Integration Interface (A):** This module facilitates the integration of external tools or services, providing additional functionalities such as database interactions, API calls, or external applications.
- **Data Processing Module (D):** This module handles further processing of input and output data, such as data cleaning, transformation, or enhancement, to support more complex use cases.
- **Prompt Collection (P):** This module contains various types of prompts and associated techniques used to interact with the LMs. The prompts may include fixed templates, dynamically generated prompts, or customized ones based on specific tasks.

In conclusion, the PF provides a modular and efficient architecture for interacting with LLMs. It separates core functionalities from advanced features, ensuring flexibility and adaptability. The Core component, which includes model collection, task management, and input/output modules, forms the foundation for efficient operations. The Extend component adds capabilities such as context management, external tool integration, data processing, and dynamic prompt generation, broadening its applicability. This structure overcomes the limitations of traditional frameworks, offering a scalable, context-aware solution for optimized LLM performance.

5.2 Workflow of Prompting Framework

We categorize the workflow for facilitating interactions between LLMs and external entities through the PF into four hierarchical layers, arranged from bottom to top: Data Level, Base Level, Execute Level, and Service Level. The PF serves as a facilitator for inter-layer communication and intra-layer scheduling. It is important to note that the interactions between different levels are non-linear. During the execution of a task, data may iteratively propagate between various layers to accomplish complex operations. The following provides a detailed description of these four levels and the role played by the PF within them. The workflow diagram is in Figure 4.

5.2.1 Data Level. The data layer serves as the foundation, interfacing with external environments to handle data transmission, preprocessing, and interactions with external sources like databases or file systems. During data ingestion, the Input Management Module (I) in the Core component performs basic preprocessing tasks, such as format conversion and error correction, ensuring data consistency across various modalities like text, images, and audio. For instance, it standardizes text data from diverse sources, removing noise and invalid characters. Meanwhile, the Data Processing Module (D) in the Extend component may extract features or classify content based on task requirements, such as identifying document types or extracting key information. This collaboration optimizes data quality and usability at the initial stage.

As for the type of input data, text refers to brief, structured or unstructured information, like commands or comments, while a document is a more complex and lengthy collection, such as reports or web pages. In the data workflow, text is processed directly, while documents require

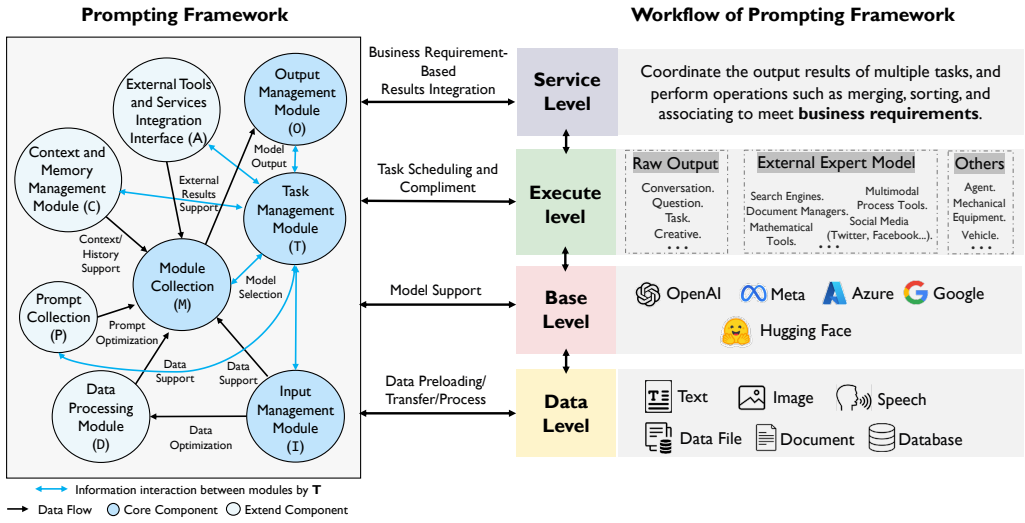


Fig. 4. The workflow for facilitating interactions between LLMs and external entities using the Prompting Framework.

additional parsing and segmentation. The I module formats text data, while the D module processes documents, breaking them into parts suitable for the model. Data is preprocessed according to type, and then routed to the base or execution layers. Complex data, like documents or multimedia (images, videos), passes through the External Tools and Services Interface (A) to expert models in the execution layer, with results integrated and sent to the base layer for LM interaction.

5.2.2 Base Level. The Base Level serves as a computational hub, connecting data and execution layers, similar to the human brain or a CPU. The Model Collection (M) manages LLMs, handling instruction reception, execution, and computations for knowledge management and decision-making. The PF coordinates data transfer and task scheduling. The Task Management Module (T) selects the appropriate model from M based on task requirements, such as choosing a translation model for language tasks or a generative model for text generation. The Context and Memory Management Module (C) ensures coherence by storing previous interactions, allowing models to reference past context in multi-turn dialogues for more relevant responses. Additionally, the Prompt Collection (P) provides the appropriate prompt templates for better-quality output.

5.2.3 Execute Level. The Execute Level is crucial for interacting with LLMs to perform real-world tasks. It coordinates task execution by using the PF, constructing tasks, and responding to model configurations. The level consists of three components: First, it uses LLMs' raw outputs from the Base Level to complete tasks autonomously. Second, it invokes external specialized models when LLMs alone can't handle certain aspects of a task. Third, it integrates LLMs with higher-order models (e.g., robotic arms, autonomous agents) to push the boundaries of AGI. The External Tools and Services Integration (A) module enables flexible integration of external data during task execution. For example, in financial market analysis, it fetches real-time data from external APIs, enhancing the model's analysis. The Output Management (O) module refines the raw model outputs into structured, user-friendly formats, such as reports or natural language responses.

5.2.4 Service Level. The Service Level sits at the top of the business workflow, overseeing the management, scheduling, and integration of advanced tasks within specific domains, in coordination

Table 1. Representative Works of Prompting Frameworks

Category	Subcategory	Representative Works
The Shell of LLMs (LLM-SH)	Universal LLM-SH	Haystack [104], Semantic Kernel [128], LangChain [21], Griptape [47], PromptFlow [51], LLM-chain [121], LinGoose [48], LLMStack [26], OpenDAN [97], Hyv [123], LangStream [60], Floom [42], Langroid [59], ModelFusion [88], Flappy [41], Dify [34]
	Domain-Specific LLM-SH	LlamaIndex [66], embedchain [120], AgentVerse [24], SuperAGI [126], Txtai [78], AutoChain [43], TermGPT [117], Botpress [12], MindSQL [87], LLMCompiler [54], Swiss Army Llama [71], QA-Pilot [107], Chainlit [19], AutoRAG [3], MemFree [77], QAnything [108]
Language for Interaction with LLMs (LLM-LANG)	Programming LLM-LANG	LMQL [10], SGLang [152], Magentic [75]
	Pseudocode LLM-LANG	PromptLang [115], SudoLang [39], GPT-jargon [14]
Output Restrictors of LLMs (LLM-RSTR)	Content LLM-RSTR	NeMo-Guardrails [92], Guardrails [118]
	Structure LLM-RSTR	Guidance [80], Promptify [100], ReLLM [113], TypeChat [81]

with the PF. It enhances the functionality of general services by interacting with the framework, particularly in key sectors like education, healthcare, e-commerce, law, and finance. In collaboration with the Execute Level, the Service Level integrates and coordinates the results of multiple tasks. For example, in complex business processes with several subtasks, it consolidates and processes the results according to business rules, ensuring a comprehensive output that meets the requirements. This hierarchical processing ensures that initial model outputs are effectively transformed and optimized for the final user or application.

5.3 Taxonomy of Prompting Framework

The current PF can be categorized into three types based on their technical features, design objectives, and application scenarios: The Shell of LLMs (LLM-SH), Language for Interaction with LLMs (LLM-LANG), and Output Restrictors of LLMs (LLM-RSTR). These types differ in the composition of extendable components. LLM-SH acts as a shell or interface, facilitating LLM interaction with external systems and enhancing communication between LLMs, users, and external models. LLM-LANG focuses on creating a concise language for user interaction with LLMs. LLM-RSTR, by controlling the generation process, ensures higher quality and more aligned outputs. In practice, these frameworks are often compatible and can be used together within the same task-solving process, depending on the specific requirements.

5.3.1 The Shell of LLMs (LLM-SH). The shell of LLMs (LLM-SH) is a PF designed to enhance LLMs by enabling access to external tools and knowledge sources. In traditional computing, a shell acts as a command-line interpreter, facilitating communication between the user and the operating system. Similarly, LLM-SH expands the functionality of LLMs, enabling them to perform not only basic NLP tasks like question answering and sentiment analysis but also advanced functions across diverse fields such as healthcare, education, finance, and computer science. These tasks include image analysis, object detection, mathematical computations, database access, and social media interactions. LLM-SH simplifies complex interactions, improving usability, scalability, and customization to meet specific needs across various domains. Table 1 presents classifications and representative works by category. While we include several examples of “Agents,” this article focuses on the Agent framework rather than individual Agents. LLM-based Agents, such as AutoGPT and BabyAGI [90], emphasize goal-oriented planning and autonomous execution, with key contributions in task acceptance, comprehension, decision-making, and execution. In contrast, Agent frameworks like SuperAGI [126] and AgentVerse [24] focus on customizing, building, managing, and running Autonomous Agents based on user needs. In short, LLM-based Agents are user-centric products, while Agent frameworks are tools for building and maintaining Agents.

The key distinction of LLM-SH from other PFs is its inclusion of the full Extend component, with a focus on the External Tools and Services Integration Interface (A) and the Context and Memory

Management Module (C). LLM-SH has two main variants: Universal LLM-SH, designed for broad application across domains, and Domain-Specific LLM-SH, tailored for specific areas like agent development, data processing, or chatbot creation.

Universal LLM-SH is designed with the intention of accommodating a wide range of application scenarios and domains, offering extensive functionality and flexibility to meet the diverse needs of users, which typically possess higher generality and scalability. Representative works include Haystack, Semantic Kernel, LangChain, Griptape [47], PromptFlow [51], LLM-chain [121], Lin-Goose [48], LLMStack [26], OpenDAN [97], Hyv [123], LangStream [60], Floom [42], Langroid [59], ModelFusion [88], Flappy [41], Dify [34].

Domain-Specific LLM-SH is tailored for specific domains or applications, optimized for better performance and efficiency in relevant tasks. Examples include LlamaIndex [66], Txtai [78], and Swiss Army Llama [71], which support building LLM-based applications. For managing autonomous agents, AgentVerse [24], SuperAGI [126], and AutoChain [43] are notable. In bot creation, Embedchain [120] and Botpress [12] are key tools. To improve parallel task handling, LLMCompiler [54] stands out. For code completion, QA-Pilot [107] is significant, while MindSQL [87] and MemFree [77] enhance database and knowledge base integration. TermGPT [117] enables LLMs to plan and execute terminal commands.

5.3.2 Language for Interaction with LLMs (LLM-LANG). Language for Interaction with LLMs (LLM-LANG) is an innovative type of PF designed to facilitate more concise, direct, and compact interactions with LLMs by introducing a specialized language for programming. Unlike traditional natural language prompts, which can be verbose and inaccurate for complex tasks, LLM-LANG uses a structured format that enhances reasoning and supports advanced methods like chain-of-thought and decision trees. It also improves control flow.

LLM-LANG stands out by focusing on optimizing the Input Management Module (I) and Data Processing Module (D). LLM-LANG has two main forms: Programming LLM-LANG and Pseudocode LLM-LANG. The key difference is that Programming LLM-LANG uses formal programming languages with strict syntax rules, requiring compilation or interpretation to produce executable code. In contrast, Pseudocode LLM-LANG combines natural language with structured code, offering a simpler, more intuitive way to describe algorithms with fewer formal constraints. While Programming LLM-LANG is more powerful for handling tasks and control flow, it comes with a steeper learning curve compared to Pseudocode LLM-LANG. Each approach has its own advantages.

Programming LLM-LANG focuses on designing a new programmable language for interacting with LLMs, simulating the syntax and structure of existing programming languages. It integrates front-end interactivity with back-end modules to optimize capabilities. By combining query languages with LLM prompts, it transforms simple text inputs into a hybrid of natural language and programming scripts, enhancing interaction intuitiveness. These composite prompts embed constraints and control flows into the LLM's parsing and output generation, reducing resource-intensive calls. For instance, LMQL [10] blends natural language with programming languages for flexible LLM task execution, while SGLang [152] uses a Python-based front-end to guide reasoning and token parallelization, improving inference efficiency and performance across tasks.

Pseudocode LLM-LANG is a more open-ended form that flexibly combines natural language with structured coding, relying on the inherent capabilities of LLMs. Pseudocode LLM-LANG leverages LLMs' capabilities while addressing issues with awkward interactions or inaccurate prompts. It offers a standardized, easy-to-understand structure that facilitates effective communication with LLMs, providing verified templates or background explanations. Representative works in this category include PromptLang [115], SudoLang [39], and gpt-jargon [14].

5.3.3 Output Restrictors of LLMs (LLM-RSTR). Output Restrictors of LLMs (LLM-RSTR) are a type of PF designed to enable controlled generation by LLMs. The controlled generation problem with LLMs pertains to how to ensure that the generated text meets specific requirements, constraints, or demands, adapting to various application scenarios and professional domains. This involves control over multiple aspects such as semantic content, output structure, and semantic style. Currently, due to the uncontrolled nature of LLMs, the generated natural language text tends to be unstructured. Additionally, generated text may contain potential risks like bias, misinformation, or inappropriate content. LLMs also struggle with off-topic responses and maintaining consistency with predefined requirements. However, the application of LLM-RSTR can effectively alleviate these issues. Unlike other PFs, LLM-RSTR emphasizes the design and performance of the Output Management (O) and Data Processing (D) modules. LLM-RSTR primarily focuses on controlled generation from two perspectives: Content Control, referred to as Content LLM-RSTR, and Structure Control, referred to as Structure LLM-RSTR.

Content LLM-RSTR aims to control LLM-generated content in three key areas: privacy, security, and alignment with the topic and accuracy. It ensures that user-provided sensitive information is protected, filters out unsafe or biased content (e.g., gender, race, political biases), and prevents the generation of false or misleading information, ensuring the output is accurate and credible. The generated text should align with the user's topic or requirements to ensure its relevance and usefulness. High accuracy is especially important in domain-specific applications such as medicine, law, and science. Notable works in this area include NeMo-Guardrails [92] and Guardrails [118].

Structure LLM-RSTR focuses on improving information processing, data management, and decision support by enabling both computers and humans to better handle and utilize information. It is particularly relevant in tasks like database management, SEO, NLP, information extraction, and data mining. Since unstructured LLM output is difficult to use in applications, guiding LLMs to produce structured output is essential. Traditional methods require complex and time-consuming templates, but Structure LLM-RSTR simplifies this process, allowing users to interact more directly with LLMs and receive clear, structured outputs. Representative works include Guidance, Promptify [100], ReLLM [113], and TypeChat [81].

5.4 Crucial Component in the Construction of Prompting Framework

The rapid development of LLMs and the PF has not only led to new technologies but also revitalized traditional methods. After defining the PF and offering a comparative analysis, this section highlights a key issue: the essential components for building a PF. Specifically, we focus on two modules in the Extend component: Prompt Collection (P) and Context and Memory Management (C).

5.4.1 Prompt Datasets and Template-Based Generator. This module contains various types of prompts and associated techniques used to interact with the LMs. The prompts may include fixed templates, dynamically generated prompts, or customized ones based on specific tasks. In general, the prompt is the most direct key factor determining the quality of interactions with LLMs, the contents and examples of representative prompt dataset and template-based generator are shown in Table 5 in appendix. For example, Awesome-ChatGPT-Prompts [1] is an open-source website and application created by JavaScript developer Fatih Kadir Akın, which contains over 160 prompt templates for ChatGPT, allowing it to mimic a Linux terminal, JavaScript console, Excel page, and more. These prompts have been collected from exemplary real-world use cases. PromptSource [4] is a toolkit for creating, sharing, and using natural language prompts. PromptSource allows for the use of thousands of existing and newly created prompts, which are stored in separate structured files

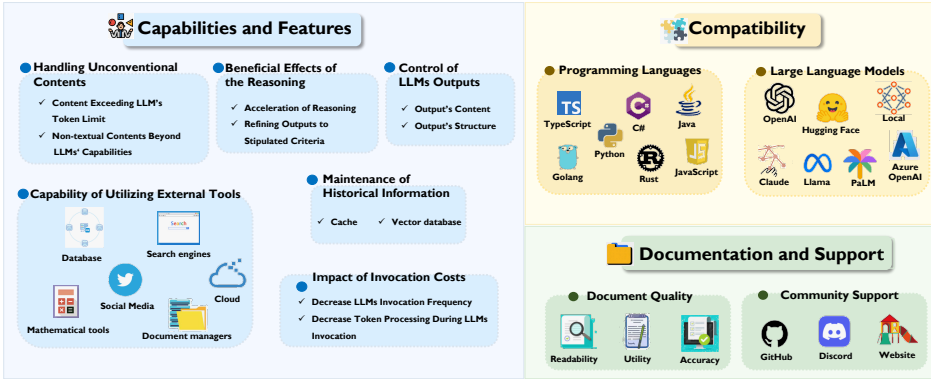


Fig. 5. The dimensions and metrics for comparative analysis with three macro dimensions of compatibility, capabilities and features, as well as documentation and support.

and written in a simple template language called Jinja. Gpt-Prompt-Engineer [44], PromptHub [106], and Promptify [106] are popular template-based generators, which directly generate high-quality prompts from the user's raw input based on the built-in templates.

5.4.2 Vector Database. The AIGC revolution introduces challenges in managing unstructured data like images, videos, and text. Vector databases address this by storing vector embeddings, which are crucial for utilizing LLMs. For example, OpenAI's Retrieval plugin uses vector databases to retrieve relevant document excerpts. A vector database stores numerical values representing features or attributes as vectors. Vectorization transforms discrete data (e.g., images, text) into continuous vector spaces, allowing semantically similar data to be grouped together. These vectors, or embeddings, reside in the embedding space. Vector databases are optimized for managing and querying large-scale vector data using similarity-based queries. Applications include image search, music recommendations, and text classification. In recent years, specialized vector databases have emerged. Milvus [130] is the first true vector database, with over 1000 enterprise users. Pinecone [105] offers speed, scalability, and integration with OpenAI's Embedding API, focusing on machine learning applications. Weaviate [35] supports billions of vectors and has a ChatGPT plugin recognized by OpenAI. The key difference between Weaviate and Pinecone is their service management: Pinecone fully handles storage and resources, often via AWS or GCP, while Weaviate allows self-hosting for more control, though with higher time investment.

6 Comparisons and Challenges

In this section, we compare existing PFs across various dimensions and analyze the challenges they face in development, implementation, and future advancements. The comparative analysis is based on the dimensions and metrics shown in Figure 5, with the capability matrix of mainstream frameworks presented in Figure 6.

6.1 Comparative Analysis of Prompting Frameworks

We have examined the three macro dimensions of compatibility, capabilities and features, as well as documentation and support. Within these dimensions, we have focused on more detailed key issues, providing a comprehensive analysis and comparison of existing PFs, which offer systematic experiences and guidelines for developers and users in their practical adoption of PFs and for further advancements.

Category	Subcategory	Representative Works	Handling Unconventional Input		Beneficial Effects of the Reasoning		Control of LLMs Outputs		Impact of Invocation Costs		Maintenance of Historical Information	Capability of Utilizing External Tools
			Content Exceeding LLM's Token Limit	Non-textual Contents Beyond LLMs' Capabilities	Acceleration of Reasoning	Refining Outputs to Stipulated Criteria	Output's Content	Output's Structure	Decrease LLMs Invocation Frequency	Decrease Token Processing During LLMs Invocation		
The Shell of LLMs (LLM-SH)	Universal LLM-SH	Haystack	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Semantic Kernel	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		LangChain	✓	△	✓	✓	✓	✓	△	✓	✓	△
		Griptape	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		PromptFlow	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		LLM-chain	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		LinGoose	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		LLMStack	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		OpenDAN	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Hvy	✗	△	✓	✓	✓	✓	✗	✓	✓	△
		LangStream	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Floom	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Langroid	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		ModelFusion	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Flappy	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Dify	✓	△	✓	✓	✓	✓	✗	✓	✓	△
	Domain-Specific LLM-SH	Llamaindex	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		embedchain	✓	△	△	✓	✓	✓	✗	✓	✓	△
		AgentVerse	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		SuperAGI	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Txtai	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		AutoChain	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		TermGPT	✗	△	△	✓	✓	✓	✗	✗	✓	✗
		Botpress	✓	△	✓	✓	✓	✓	✗	✗	✓	△
		MindSQL	✓	△	✓	✓	✓	✓	✗	✗	✓	△
		LLMCompiler	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Swiss Army Llama	✓	△	△	✓	✓	✓	✗	✓	✓	△
		QA-Pilot	✓	△	✓	✓	✓	✓	✗	✗	✓	△
		Chainlit	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		AutoRAG	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		MemFree	✓	△	✓	✓	✓	✓	✗	✓	✓	△
		Qanything	✓	△	✓	✓	✓	✓	✗	✓	✓	△
Language for Interaction with LLMs (LLM-LANG)	Programming LLM-LANG	LMQL	△	△	✓	✓	✓	✓	✓	△	△	△
		SQLang	✓	△	✓	✓	✓	✓	✓	△	△	△
		Magentic	✓	△	✓	✓	✓	✓	✗	✗	△	△
	Pseudocode LLM-LANG	PromptLang	✗	✗	✓	✓	✓	✓	△	△	✗	✗
		SudoLang	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗
Output Restrictors of LLMs (LLM-RSTR)	Content LLM-RSTR	GPT-Jargon	✗	✗	✓	✓	✓	✓	✗	✗	✗	✗
		NeMo-Guardrails	✓	△	✗	✓	✓	✓	✗	✓	△	△
		Guardrails	✓	✗	✗	✓	✓	✓	✗	✓	✗	✗
	Structure LLM-RSTR	Guidance	✓	✗	△	✓	✓	✓	✗	✓	△	△
		Promptify	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗
		ReLLM	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗
		TypeChat	✓	✗	✗	✓	✓	✓	△	✓	✗	△

Fig. 6. The capability matrix of representative prompting frameworks.

6.1.1 Compatibility. Compatibility refers to the adaptability and interoperability of systems, software, hardware, or other components in various environments or conditions. We primarily investigate the compatibility of the PF with programming languages and its compatibility with LLMs.

Compatibility of Programming Languages. Due to different development team preferences and project requirements, PFs are typically designed to consider the use of multiple programming languages by developers. As a result, various PFs often provide interfaces supporting one or multiple mainstream programming languages to allow developers to flexibly choose and interact with languages in different projects and environments, thereby enhancing development efficiency and adaptability. The currently available PF provides comprehensive coverage of programming languages, including Python, Java, JavaScript, C#, C++, Go (Golang), Rust, and TypeScript.

Comparative Analysis. Currently, the PF is still in its early stages of rapid development. Language support tends to begin with an implementation in a specific language and then expand to support a wider range of programming languages, accommodating more use cases and domains. Overall, LLM-SH is designed to be more adaptable to a broader range of scenarios and to support a richer set of tools. Consequently, LLM-SH exhibits stronger compatibility with programming languages compared to LLM-RSTR and LLM-LANG. For instance, Txtai, within LLM-SH, offers impressive support for five mainstream programming languages: Python, Java, Rust, Golang, and JavaScript. Meanwhile, LangChain and Semantic Kernel can each support three programming languages: Python, TypeScript, and JavaScript, as well as C#, Python, and Java, respectively. Most PFs tend to focus on supporting one mainstream programming language, with Python and TypeScript

being the primary choices. For instance, in LLM-SH, tools like Haystack, Griptape, PromptFlow, LLMStack, OpenDAN, AutoChain, TermGPT, and in LLM-LANG, LMQL, as well as in LLM-RSTR, NeMo-Guardrails, Guardrails, Guidance, Promptify, and ReLLM primarily support Python. On the other hand, Hyv and botpress belonging to LLM-SH, and TypeChat belonging to LLM-RSTR are specifically designed for TypeScript. LlamaIndex and embedchain in LLM-SH support both Python and TypeScript. Furthermore, LLM-chain is exclusively focused on Rust, while LinGoose is tailored for the Golang. As for Pseudocode LLM-LANG, its intent is to create a new language to simplify interaction with LLMs, so SudoLang, PromptLang, and gpt-jargon support only the syntax of the language designed within the framework and natural language.

◇ **Summary:** In terms of compatibility with programming languages, LLM-SH surpasses LLM-RSTR, which in turn surpasses LLM-LANG. Most prompting frameworks typically support only one mainstream programming language, particularly Python.

◇ **Practical Recommendations:** For projects requiring support for niche or multiple programming languages, LLM-SH offers the widest compatibility, including Python, Java, Rust, Golang, and JavaScript. For Python or TypeScript-based projects, LLM-SH, LLM-RSTR, and LLM-LANG all provide strong support.

Compatibility of LLMs. The original intention of the PF is to establish a medium for interaction between external and LLMs. Therefore, one of the key requirements for the PF is its compatibility with LLMs, which means that the PF should seamlessly integrate with various types of LLMs and handle their inputs and outputs correctly. In other words, PFs with better compatibility generally offer a unified interface, which enables users to use the different PFs with corresponding instructions to interact with LLMs, without requiring additional handling of specific details. The current PF supports various LLMs, primarily including the previously mentioned OpenAI API, Azure OpenAI API, HuggingFace API, as well as other APIs such as Llama API, Anthropic, ERNIE-Bot, Google PaLM, and so on.

Comparative Analysis. Currently, support for LLMs within PFs is in a phase of rapid evolution due to the continuous emergence of new LLMs. Developers need to swiftly integrate these models into PFs once they are familiar with their functionalities and usage. It's worth noting that when we refer to LLMs compatibility, we mean the ability to seamlessly integrate models into PFs without the need for complex additional operations or coding, which implies direct utilization of models in PFs through modifications to certain configurations, using the LLMs in their native supported form, rather than relying on additional plugins or frameworks. Models that are natively supported by PFs tend to integrate better with the framework, fully leveraging the model's capabilities and minimizing unexpected bugs during programming. At present, all PFs offer good support for prominent APIs released by OpenAI, namely text-davinci-003, gpt-3.5-turbo, chatgpt, gpt-4, and gpt-4o. In other words, PFs were originally introduced to facilitate user interactions with these mainstream APIs. Pseudocode LLM-LANG is a special case where this mutation-free pseudocode language can work well with any interactive interface-based LLMs. Consequently, SudoLang, PromptLang, and gpt-jargon can be compatible with almost all interactive LLMs.

We also present the maximum tokens supported and the pricing of mainstream LLMs, as shown in Table 2. The support for Hugging Face API and local private LLMs in current mainstream PFs is still under development, showing a clear trend toward compatibility with a broader range of online or offline LLMs. For instance, LLM-SH frameworks such as LangChain, Semantic Kernel, Haystack, Griptape, PromptFlow, LinGoose, LlamaIndex, embedchain, Txtai, and AutoChain, as well as

LLM-RSTR frameworks like TypeChat and Promptify, provide robust support for this. Additionally, for other popular APIs, including emerging LLM APIs like Anthropic's Claude, Llama API, Google PaLM, Gemini, and Qwen, LLM-SH frameworks such as LangChain, Semantic Kernel, Haystack, Griptape, Txtai, LlamaIndex, and embedchain also offer strong compatibility. It is important to note that the support for a wider variety of LLMs in current LLM-LANG and LLM-RSTR frameworks is generally limited. However, this does not imply that LLM-LANG and LLM-RSTR are inferior to LLM-SH in practical use. The differences in focus and strengths arise from the distinct design goals of these frameworks. Moreover, thanks to the modular nature of PFs, LLM-LANG frameworks like SALang and LLM-RSTR frameworks such as Guidance can integrate with LLM-SH frameworks like LangChain to support a broader range of mainstream LLMs. This flexible, integrated development approach allows users to implement advanced features while minimizing debugging costs associated with integrating different toolsets, leading to more efficient development.

◇ **Summary:** All prompting frameworks support OpenAI's API well. LLM-SH offers the best compatibility with LLMs, while LLM-LANG, based on pseudocode, does not require compilation for interactive interfaces. LLM-LANG and LLM-RSTR can overcome model compatibility issues by integrating with an LLM-SH framework.

◇ **Practical Recommendations:** For compatibility with various online and offline LLMs, LLM-SH (e.g., LangChain) is the best choice, which offers broad support and is improving compatibility with Hugging Face and private LLMs. While LLM-LANG and LLM-RSTR have limited support, integrating them with LLM-SH can enhance compatibility, streamline development, and reduce debugging efforts.

6.1.2 Capabilities and Features. Capabilities and features are crucial when comparing different PFs, as they directly determine the framework's ability and flexibility in addressing problems and meeting user requirements. We elaborate on various crucial stages of the interaction between LLMs and the PF, including data preprocessing, reasoning process, output control, cost considerations, tool learning, and information maintenance. The comparison dimensions of capabilities and features we have enumerated can be employed not only for analyzing the strengths and limitations of diverse PFs but also as evaluation metrics for forthcoming tasks in assessing the PFs.

Capability of Handling Unconven-

tional Contents. The ability of PFs to assist LLMs in handling unconventional content primarily manifests in two aspects. Firstly, the PF helps LLMs deal with content that exceeds the token limit, and secondly, the PF empowers LLMs to process non-textual content formats that go beyond their inherent capabilities. Usually, LLMs have a token limit determined by the model's architecture and memory constraints. For example, GPT-3.5-turbo and gpt-4-turbo have a maximum input length

Table 2. The Maximum Tokens Supported and Pricing of Mainstream LLMs

Model	Developer	Max Token (Context Window)	Cost / 1M tokens	
			Input	Output
gpt-4o	OpenAI	128,000	\$2.50	\$10.00
gpt-4o-mini	OpenAI	128,000	\$0.150	\$0.600
o1	OpenAI	200,000	\$15.00	\$60.00
o1-mini	OpenAI	128,000	\$3.00	\$12.00
gpt-4-turbo	OpenAI	128,000	\$10.00	\$30.00
gpt-4	OpenAI	8,192	\$30.00	\$60.00
gpt-3.5-turbo	OpenAI	16,385	\$0.50	\$1.50
Gemini 1.5 pro	Google	128,000	\$1.25	\$5.00
Gemini 1.0 pro	Google	32,000	\$0.50	\$1.50
Claude 3.5 Sonnet	Anthropic	200,000	\$3.00	\$15.00
Claude 3.5 Haiku	Anthropic	200,000	\$0.80	\$4.00
Claude 3 Opus	Anthropic	200,000	\$15.00	\$75.00
qwen-turbo	Alibaba	100,000	\$0.30	\$2.00
qwen-max	Alibaba	8,000	\$20.00	\$60.00
PaLM 2	Google	8,000	\$0.5	\$0.5
Llama3	MetaAI	8,192	free	free
Llama2	MetaAI	4,096	free	free
Mixtral	Mistral AI	6,400	free	free
Phi-3	Microsoft	128,000	free	free

of 16,385 and 8,192 tokens, while gpt-4o and gpt-4o-mini allows for 128,000 tokens, and OpenAI o1 allows for 200,000 tokens, which results in LLMs being unable to capture the global context of a text when dealing with extremely long documents without external support because only a portion of the text can be included. Even though the context window length has been significantly expanded compared to the previous version, processing long contexts still exposes issues related to forgetting and comprehension. The maximum tokens supported and pricing of mainstream LLMs are shown in Table 2 and the statistical diagram of the relationship between maximum context window and input token billing is shown in Figure 7. Moreover, processing such long text can lead to performance issues, especially in resource-constrained environments. LLMs need to maintain a lot of information within a single input, which may require more computational resources and time. Therefore, PFs have emerged to assist LLMs in handling complex tasks involving extremely long texts, such as machine translation, legal document analysis, sentiment analysis of lengthy novels or articles, long-context dialogue systems, knowledge graph construction, and so on. LLMs are primarily designed for processing pure textual data, making them less suitable for unconventional formats like images, audio, or videos, whose main inputs and outputs are text data. PFs effectively mitigate this limitation, enabling LLMs to handle a variety of data types, which broadens the application of LLMs to a wide range of multimedia tasks.

Comparative Analysis. In terms of the capability to handle content exceeding token limits and non-textual content, only LLM-SH can achieve these functionalities without the need for additional plugins or program calls. LLM-LANG and LLM-RSTR achieve these capabilities by making calls to LLM-SH within their framework design. However, it's important to note that such compatibility can potentially lead to unexpected bugs.

For handling content that exceeds token limits, LLM-SH empowers LLMs through methods like splitting, filtering, concatenation, or summarization. For instance, a classic approach is the “Retrieval” module in LangChain, where “Document transformers” offer pre-packaged functional functions for document splitting, composition, and filtering. In the “Chains” module, there is a package called “chains.summarize” that provides various methods for summarizing documents (PDFs, Notion pages, customer questions, etc.), including Map-Reduce, Stuff, and Refine approaches to organizing documents. By configuring parameters to design a “chain” structure differently and introducing vector databases and text embedding models, LLMs can effectively manage extremely long documents. Similarly, modules like “Summarizer” in Haystack and “Summary Engines” in GripTape can summarize long texts, and components like “PreProcessor” in Haystack and “Chunkers” in GripTape can perform text splitting and filtering for long texts.

When it comes to handling non-textual content, LLM-SH can process text extracted from non-text formats like YouTube videos or HTML files or generate multi-modal content such as images, videos, and audio, based on pure text content. However, it doesn't provide full-fledged processing of multimedia files throughout the input-output process. For instance, the “Tool Memory” module in GripTape can generate images, videos, PDFs, and other non-textual content. Similarly, the

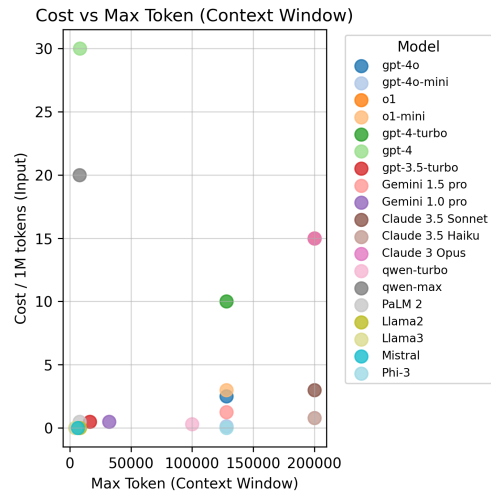


Fig. 7. Statistical diagram of the relationship between maximum context window and input token billing.

“Document loaders” module in LangChain exposes a “load” method for loading data as documents from a configured source. With the rapid development of artificial intelligence, LLMs are evolving to support multimodal capabilities. Many models originally designed for text processing, such as OpenAI’s GPT-4, now also handle multimodal data like images, videos, and audio. Existing PFs are increasingly adapting to support multimodal LLMs. For example, LangChain effectively supports multimodal models like GPT-4, Gemini, and Claude in processing diverse data types, enabling more complex and advanced tasks. Currently, only LLM-SH provides substantial support for multimodal models, while LLM-LANG and LLM-RSTR have focused less on this feature. However, integrating suitable LLM-SH models can overcome this limitation.

◆ **Summary:** Currently, only LLM-SH can effectively handle unconventional input contents, including processing content that exceeds token limits through splitting, filtering, reorganization, or summarization of long documents, as well as managing multimodal streams such as images and videos. LLM-LANG and LLM-RSTR can only partially achieve this functionality by relying on the relevant modules of LLM-SH.

◆ **Practical Recommendations:** If the application requires handling non-text content or content exceeding token limits, LLM-SH is recommended. If functionalities from LLM-LANG or LLM-RSTR are needed, consider integrating the LLM-SH model to extend compatibility and functionality.

Beneficial Effects of the Reasoning Process. The benefits of PFs for LLMs in the reasoning process are primarily evident in two capabilities: accelerating reasoning and ensuring that reasoning results meet preset requirements. In the current scenario with a surge in data volume and user usage, the speed of LLMs’ reasoning is crucial for completing tasks efficiently. Additionally, improved reasoning results enable LLMs to accomplish more work in a given unit of time, greatly facilitating the application of these powerful NLP models in tasks requiring rapid responses and high throughput. For example, it’s useful in online customer support (providing instant responses to customer queries for better user experiences), real-time financial analysis (for timely financial decision-making by analyzing news events and market data), and real-time sentiment analysis (for tracking product or brand feedback on social media in real-time).

Comparative Analysis. Most PFs aim to accelerate inference by reducing input tokens but lack targeted acceleration. Among those studied, only LMQL, SGLang (in LLM-LANG), and LLMCompiler (in LLM-SH) offer specific acceleration features, though with different approaches. LMQL accelerates inference through runtime validation. It monitors text generation and halts the process if output conditions (e.g., “excluding clouds” and “excluding birds”) are violated, prompting the model to retry from the valid part of the output. This “verify-as-you-go” method reduces waiting time by validating outputs incrementally. SGLang, combining a specialized frontend language and runtime optimizations, accelerates execution. Its frontend simplifies generation and parallel control, while the runtime incorporates RadixAttention for KV cache reuse and a compressed finite state machine for faster decoding. These features enhance throughput for various tasks, including agent control, logical reasoning, and multimodal processing. LLMCompiler applies principles from classical compilers to accelerate inference by decomposing tasks and parallelizing subtasks. It uses a function call scheduler, task acquisition unit, and parallel executor to optimize function call orchestration, applicable to both open-source and closed-source models. To ensure inference meets requirements, LLM-LANG, LLM-RSTR, and LLM-SH utilize validation templates or allow user-defined templates for standardized prompts. Features like LangChain’s “Prompts” and Haystack’s “PromptTemplate”

facilitate predefined and user-defined templates. Eager validation not only speeds up inference but also ensures compliance with output expectations.

◇ **Summary:** To enhance the inference process of LLMs, LLM-LANG, LLM-RSTR, and LLM-SH all use methods such as predefined templates or compatibility with user-defined templates to ensure that inference results better meet requirements. The LLM-LANG's LMQL and SGLang, as well as LLM-SH's LLMCompiler, are the only components that support the acceleration of the LLM runtime inference process, achieving acceleration at the intermediate expression, token, and task levels, respectively.

◇ **Practical Recommendations:** To improve the inference efficiency of LLMs, LLM-LANG's LMQL and SGLang, as well as LLM-SH's LLMCompiler, provide acceleration at the intermediate expression, token, and task levels, respectively. To ensure inference results meet predefined requirements, LLM-LANG, LLM-RSTR, and LLM-SH offer predefined templates or methods compatible with user-defined templates for more standardized and accurate inputs.

Control of LLMs Outputs. PFs play a key role in controlling LLM outputs, addressing two main concerns: enforcing fine-grained structural constraints and preventing unsafe or non-compliant content. Structured output from LLMs enhances information processing and analysis by converting natural language into a usable format, which aids in tasks like extracting key information, building knowledge graphs, and data analysis. This structured data is vital for decision-making, process optimization, and developing intelligent applications across industries. Ensuring the safety and relevance of LLM outputs is crucial for accuracy and appropriateness. For instance, content filtering on social media ensures posts comply with platform rules, while automated customer support systems focus on relevant, appropriate responses. These safety measures protect brand reputation, user rights, and improve service availability. PFs are essential for guiding LLMs to generate output that meets these standards, with applications spanning social media, e-commerce, online education, and more.

Comparative Analysis. When it comes to constraining the output structure of LLMs, LLM-SH, LLM-LANG, and Content LLM-RSTR typically provide interfaces or pre-packaged functions that, when given custom structured templates provided by users, transform LLMs' output into the corresponding structured format. For example, LangChain's Model I/O module offers "Output parsers," which are classes designed to help structure LM responses. By invoking LangChain's built-in functions and combining them with user-configured structural templates, LLM-SH's LangChain directly supports various commonly used structured output forms, such as lists, time formats, enumerations, JSON, and more. In Semantic Kernel within LLM-SH, LMQL within LLM-LANG, and SudoLang, which are designed with frameworks and syntax that allow the mixing of natural language with code, support for structured data is achieved through user-configured custom structural templates. This means that users need to design, describe, and write their own templates in the corresponding framework's custom template locations. However, in the case of Structure LLM-RSTR, a category of PFs specifically designed for structured output, they offer built-in features for structured data generation and validation and excel at supporting formats like JSON and dialogue-based data. For instance, in Guidance, you can use the "gen" command to interleave generation, prompting, and logic control in a continuous sequential flow that aligns with how the LM processes text. In TypeChat, there's the provision of Schema, a data structure used to describe

the expected format and fields of a prompt. By defining a Schema, you can validate and correct user input to ensure it adheres to the expected format and requirements.

Securing and ensuring compliance with the output content of LLMs is crucial. However, except for Content LLM-RSTR within LLM-RSTR, no other PFs address this issue. In Content LLM-RSTR, Guardrails is a Python package that adds type and quality assurance to LLMs' output, including semantic validation, such as checking for biases in generated text and errors in generated code and takes corrective measures when validation fails. Guardrails provides a file format (.rail) for executing "specifications" (user-specified structural and type information, validators, and corrective actions) on LLMs' output and offers a lightweight API call wrapper to implement these specifications on the output. NVIDIA's NeMo-Guardrails follows a similar design philosophy, ensuring control, safety, and security of LLM language generation by limiting templates and themes in conversations. NeMo-Guardrails employs a custom language to implement a three-layer protection mechanism, including Topical GuardRail for topic-related questions, Safety GuardRail for ensuring controlled responses, and Security GuardRail to protect against malicious or abusive questions. However, this approach, while practical, also introduces potential challenges such as configuration redundancy, inflexibility, operational risks, and limitations on the original conversational capabilities of LLMs.

◇ **Summary:** LLM-SH, LLM-LANG, and LLM-RSTR all offer different ways to support control over the structure of LLMs' output, but only Content LLM-RSTR within LLM-RSTR provides constraints and validation for the safety and compliance of LLMs' output content, albeit with certain potential issues in its functionality.

◇ **Practical Recommendations:** LLM-SH, LLM-LANG, and LLM-RSTR offer functions to transform LLM outputs into structured formats using custom templates, ensuring alignment with expected structures for further processing. To guarantee safety and compliance, features like Guardrails in Content LLM-RSTR help protect brand reputation, user rights, accuracy, and system usability.

Impact of Invocation Costs. Reducing the cost of LLMs invocation primarily involves two approaches: firstly, reducing the frequency of invoking LLMs, and secondly, decreasing the number of tokens processed during LLMs invocation. Utilizing LLMs services on cloud computing platforms such as AWS, Azure, and Google Cloud incurs charges for each model invocation. These costs escalate with the frequency of invocations and the extent of computational resources used. One of the significant challenges in implementing LLMs for practical production scenarios is the high cost associated with invoking LLMs, which can be burdensome for individuals and even enterprises with limited financial resources. LLMs are typically billed based on the number of tokens generated or processed, with the number of tokens subject to charges determined by the prompt and the corresponding response length. The billing unit is the "token," which can represent a word, character, or punctuation symbol. In English, one token typically corresponds to one word, but for other languages, token forms may vary due to differences in language structures. Different models have varying token pricing standards; for example, GPT-3.5 is priced at \$0.002 per 1000 tokens, while GPT-4's token price is nearly six times higher than that of GPT-3.5. The maximum tokens supported and pricing of mainstream LLMs are shown in Table 2. While the cost per individual token may appear acceptable from a computational perspective, it becomes substantial for long-term usage and complex tasks. For instance, translating large documents, especially complex technical documents across multiple languages, may require extended processing time and additional computational resources. Similarly, employing LLMs for large-scale data analysis, text mining, or information

extraction tasks, such as processing tens of thousands of news articles to extract key information, might necessitate distributed computing environments and substantial storage, resulting in high invocation costs.

Comparative Analysis. In terms of reducing the frequency of LLM invocation, LLM-SH introduces the “Memory” module that stores historical query information. Leveraging vector databases, enables a form of “learning from the past,” so that when encountering the same or similar questions, LLMs do not need to be invoked again, thus saving unnecessary computational expenses. For instance, LLM-SH’s LangChain implements a “Memory” module for storing past interactions, Haystack includes a “Module memory” module with the `InMemoryDocumentStore` class for recording and retrieving interaction content, and Griptape offers a “Conversation Memory” module with `BufferConversationMemory` functionality for constructing prompts with sliding window tasks. In contrast, LLM-LANG primarily reduces the frequency of invoking LLMs by embedding preprocessing programs and modules within the framework. Simple questions are first handled by smaller or free models, and then LLMs are utilized for more detailed processing, thus reducing the number of LLMs invocations. However, this method has limitations in broader tasks. Additionally, LLM-RSTR frameworks typically do not consider this functionality. Regarding the PF’s role in reducing the number of tokens that LLMs need to process, LLM-SH employs techniques like splitting, filtering, concatenating, or summarizing text segments to abbreviate and simplify the content to be processed. This aligns with the earlier-discussed approach of handling extremely long documents.

◇ **Summary:** Current prompting frameworks fail to effectively reduce the cost of LLM invocations. LLM-SH uses “Memory” functionality and text manipulation to minimize invocation frequency and token processing. LLM-LANG relies on limited preprocessing, while LLM-RSTR lacks this capability. Future development should focus on reducing LLM invocation costs.

◇ **Practical Recommendations:** To reduce the cost of LLM calls, the “memory” module in the LLM-SH framework can minimize redundant calls, the preprocessing module in the LLM-LANG framework can handle simple tasks before calling the LLM for more complex ones, and text manipulation techniques in the LLM-SH framework can reduce the number of tokens processed. These methods help lower computational costs and improve the cost-effectiveness of LLM applications.

Capability of Utilizing External Tools. The ability to use tools is one of the most significant distinctions between humans and other species. Currently, a crucial limitation of LLMs in practical applications is their inability to use external tools like humans. However, the advent of PFs effectively mitigates this deficiency. These advancements allow LLMs to interact with external systems and enhance their functionality, enabling them to go beyond pure language understanding and generation. Typically, LLMs are generalized models, meaning they are trained to perform a wide range of tasks but do not specialize in any single one. Therefore, the capability for LLMs to utilize external tools holds significant importance for enhancing their functionality, adapting to specific tasks and domain requirements, providing access to more data and resource support, and meeting compliance and security demands. For example, in custom text generation tasks, combining external plugins with LLMs allows the generation of industry-specific news reports, creative writing, or legal documents. This is particularly valuable in industries where nuanced knowledge and adherence to specific standards are essential. In complex data query tasks, integrating LLMs with database query plugins opens up new possibilities, allowing the model to respond to intricate business queries

such as generating sales reports, managing inventory, or retrieving medical records. Additionally, by connecting LLMs with multimodal processing plugins, LLMs can analyze and generate content that includes not just text, but also images, audio, and video elements. This capability is particularly useful for tasks such as creating dynamic social media posts, advertisements, or educational content that requires a blend of textual and visual information.

Comparative Analysis. In terms of enabling LLMs to use external tools, LLM-SH outperforms LLM-RSTR and LLM-LANG due to its design focus. LLM-SH can support the integration of LLMs with a wide range of tools spanning various domains, such as web browsers, databases, e-mail clients, image processing tools, speech recognition engines, code processors, and more. For instance, LangChain’s Agents module offers a Toolkit function in which a collection of tools designed for specific tasks and conveniently loaded methods are available. These tools encompass Gmail, GitHub, SQL Databases, Vectorstore, and various Natural Language APIs. GripTape utilizes the Tools module’s TextToolMemory to enhance the output of all tools, while also allowing users to perform different degrees of customization at the structural, task, or tool activity levels. TextToolMemory empowers LLMs to interact with the external world, generating non-textual content such as images, videos, PDFs, and others. In contrast, LLM-LANG and LLM-RSTR provide limited support for only the most commonly used external tools. For instance, LMQL in LLM-LANG offers services restricted to calculators and Wikipedia tools, while Guidance in LLM-RSTR offers integration with the Bing search engine.

◇ **Summary:** Current prompting frameworks for integrating LLMs with external tools are still in the early stages. LLM-SH enables integration with a range of tools, while LLM-LANG and LLM-RSTR offer only limited support.

◇ **Practical Recommendations:** To improve LLM practicality, prioritize the LLM-SH framework, which supports various tools like browsers, databases, and email clients. This allows LLMs to handle complex tasks such as generating industry reports and performing multimodal analysis. In contrast, LLM-LANG and LLM-RSTR support only a few basic tools, making LLM-SH the better choice for domain-specific tasks.

Maintenance of Historical Information. Maintenance of historical interaction information is crucial for LLMs as it provides them with the ability to store, retrieve, and reference information. This capability aids in better understanding context, maintaining coherence in complex tasks and long texts, and benefiting from past computations. However, apart from chat models like ChatGPT, which can record some history within the same conversation interface (with no sharing of information across different interfaces), LLMs typically process only the current context information provided in the prompt during service. This limitation is inconvenient for users as historical interaction information not only enhances interactions with LLMs, reducing the need for repetitive work but also stimulates non-dialogue LLMs like GPT-3.5 in chat tasks. PFs address this need by offering “Memory” systems, supporting two fundamental operations: reading and writing. This enables the storage of historical interaction information and its utilization through queries, searches, and retrieval. PFs empower LLMs with the ability to maintain historical interaction information, aiding in context understanding, information retrieval, multi-turn question-answering, document reading, error detection and correction using memory, caching intermediate computation results, and storing user preferences, historical behavior, or personalized information.

Comparative Analysis. Regarding the maintenance of historical interaction information, LLM-SH excels due to its comprehensive Memory system, which allows integration with various tools across

different domains. Taking LangChain as an example, the “Memory” can store past chat messages, queries, and results, effectively adding memory to the system. This storage function can be used independently or seamlessly integrated into various chains. Moreover, LangChain supports the use of embedded models and vector databases to store, query, and maintain historical information. During a single run, the system interacts with “Memory” twice, once when providing a query. The prompt includes two parts, one directly from user input, and the other possibly extracted from information stored in memory. The second interaction records the current interaction in memory for future queries and retrieval. LLM-SH’s Memory system usually offers multiple ways to manage short-term memory, such as ConversationMemory and VectorStore-backed Memory. ConversationMemory summarizes ongoing conversations and stores the current summary in memory, passing it as short-term memory to LLMs in new rounds of conversation. This compression of historical conversation information is particularly useful for lengthy conversations. On the other hand, VectorStore-backed Memory stores all past conversations in vector form in a vector database. In each new conversation round, it matches the user’s input against the vector database to find the most similar K sets of conversations. LLM-LANG and LLM-RSTR typically achieve historical information maintenance by invoking the corresponding functional modules in LLM-SH. An interesting feature in LLM-LANG is LMQL’s “Caching Layer,” implemented as a tree-like data structure. This layer caches all model outputs, including logs and historical information, to enable more efficient exploration of the LLM’s token space during runtime, even in the presence of multiple variables, constraints, and tool enhancements. The cache can be seen as a tree with only appendices, explored during query execution and expanded based on query code, constraint conditions, and inferred execution scenarios.

◆ **Summary:** LLM-SH provides a comprehensive memory system for storing interaction history, while LLM-LANG and LLM-RSTR rely on LLM-SH’s modules to achieve this.

◆ **Practical Recommendations:** To maintain interaction history and improve LLM capabilities, we recommend using LLM-SH for storing and retrieving data, essential for dialogue coherence and context understanding. Integrating embedding models and vector databases further enhances performance in complex tasks and long-text scenarios, improving user experience.

6.1.3 Documentation and Support. Document quality and community support are crucial for a PF. Document integrity refers to the accuracy, readability, and completeness of technical documentation, which assesses whether the development team provides detailed, clear, and comprehensive documentation to facilitate user learning and usage. Community support involves the presence of an active developer community around the framework, allowing users to receive timely assistance and feedback.

Document Completeness. The investigated PFs offer three primary deployment methods. The first method is command-line installation, where Python-based frameworks use commands like “pip install” or “conda install,” and JavaScript-based frameworks use “npm install” for deployment. This method is available for almost all PFs. The second method is a custom interactive user interface, providing a playground that allows users to interact with the framework directly through a user-friendly interface without the need for installation, for example, LMQL in LLM-LANG. The third method is the “open-and-use” approach, where deployment and execution are done directly on the interface provided by LLMs, offering the most straightforward and simple operation. SudoLang in LLM-LANG is an example. Overall, the PFs studied provide comprehensive technical documentation

that covers almost all features and use cases. They also offer detailed example code, tutorials, and operation guides, making it easy for users to get started, understand, and use these frameworks. These technical documents are readily accessible on GitHub. Additionally, some frameworks have created well-designed web pages with high readability and attractiveness, such as LangChain in LLM-SH.

Document Readability. At present, the technical documentation of existing PFs is comprehensive and detailed but may lack readability. The rapid proliferation of LLMs and their derivatives has led to a need for frequent updates to the technical documentation. These updates may occur daily to accommodate more products and features. The introduction of new features can sometimes render old ones unusable or introduce bugs, causing significant challenges for users and developers. Furthermore, the continuous addition of features and product introductions can make the documentation structure complex and the content increasingly lengthy. This can lead to a situation where the documentation becomes overwhelming and resembles a “code mountain” rather than maintaining the clarity and simplicity of its initial state, which can be frustrating for users. In summary, the dynamic nature of LLMs and their associated frameworks necessitates frequent documentation updates, but these updates must be managed carefully to maintain clarity and usability for users and developers.

Community Support. Regarding community support and user feedback channels, the development teams behind these PFs typically offer Discord online community services. Users can ask questions, share experiences, and engage with other users in these communities. Furthermore, they maintain official support teams on X and have dedicated communication topics (tags) for users to provide feedback, suggestions, and opinions, thus facilitating framework improvements. Additionally, they provide communication spaces on GitHub for users and developers.

◇ **Summary:** Existing prompting frameworks provide various deployment options, including command-line installation, customizable interfaces, and “out-of-the-box” solutions. While GitHub offers technical documentation and examples, frequent updates and complex content can impact readability.

◇ **Practical Recommendations:** Choose prompting frameworks with clear, user-friendly documentation, such as LangChain in LLM-SH, known for its intuitive design. Also, prioritize frameworks with active community support through channels like Discord, X, and GitHub to resolve issues quickly and foster ongoing improvement.

6.1.4 Quantitative Analysis. Efficiency is crucial for the usability and cost-effectiveness of PFs, especially when processing large-scale data or complex tasks. An efficient framework provides faster, more accurate results while reducing resource consumption, which benefits both businesses and developers. This study conducts a quantitative analysis of representative PFs, comparing their accuracy and efficiency across different tasks to offer practical recommendations.

Despite many frameworks supporting OpenAI models for comparative testing, a systematic evaluation of their correctness, efficiency, and resource use is challenging due to three key issues: (1) **Fair Testing Methodology.** Most frameworks inherit core functions from LangChain (e.g., API calls, browser interactions, database access), which influence execution time. This interdependence makes it difficult to isolate individual framework features and develop a fair evaluation method. (2) **Impact of Underlying APIs on Simple Tasks.** For simple tasks, performance largely depends on the underlying APIs, making it hard to differentiate frameworks based on their own architecture or algorithms. (3) **Lack of Objective Metrics for Complex Tasks.** Complex tasks require assessment across multiple subjective factors—such as completeness, reasoning, and interpretability—making

objective evaluation difficult. To address these challenges, we selected diverse tasks for a case study to perform a quantitative analysis.

Experimental Setup. To ensure the accuracy and fairness of the tests, all experiments were conducted on an Apple MacBook Pro with macOS 12.1, 32 GiB RAM, and a 10-core M1 Pro CPU. The results were averaged over multiple runs. GPT-4 models were deployed remotely, and interactions were conducted via APIs. As a result, the latency observed in our results may be influenced by internet conditions; however, we ensured that all techniques were assessed in the same environment. To obtain accurate answers, we selected questions that have standard answers, using the HotpotQA [144] dataset for testing. HotpotQA contains 1,500 user queries designed to compare two distinct entities based on specific aspects, such as “Are both Duke Energy and Affiliated Managers Group based in Massachusetts?”. Additionally, our focus was primarily on PFs designed for Python.

CaseStudy 1: Question Answering System based on Prompt Templates. Initially, we tested the basic capabilities of different PFs using the simplest task—inquiring with the same question template across simple chatbots built with different PFs (which include remote calls to LLM APIs) to obtain answers. Additional information was provided in text form and included in the prompt. We analyzed the performance of different frameworks by calculating the accuracy of their responses and the time taken for each task. The results are shown in Table 3. We tested representative PFs based on the template of ReAct [146]. Since most LLM-SH models, including Universal LLM-SH, rely on Langchain for their underlying API calls, and considering the similarity in task flow and difficulty of setup, we selected five frameworks for evaluation: Langchain, Semantic Kernel, Haystack, Griptape, and LLM-chain. To eliminate the effects of network and LLM response variability, we performed three rounds of testing for each framework. The results indicated that the accuracy and time consumption of the different frameworks were virtually identical. The latency difference was no greater than 0.3 seconds, and the accuracy difference was less than 0.5%. Thus, we conclude that for simple tasks requiring minimal module involvement, and when using the same underlying LLM APIs, the differences in accuracy and time between frameworks are negligible. Consequently, users can choose a framework based on their practical preferences without significant constraints.

Table 3. The Accuracy and Latency Results of Case Study 1

LLM-SH	Accuracy	Latency
LangChain	61.78%	4.13s
Semantic Kernel	61.81%	4.22s
Haystack	61.75%	4.18s
Griptape	61.70%	4.21s
LLM-chain	61.82%	4.19s

◇ **Practical Recommendations:** For simple tasks with minimal module interactions, the differences in accuracy and response time between frameworks using the same underlying LLM API are negligible. Users can prioritize their preferences—such as community support, documentation, or interface design—without worrying about significant performance differences.

CaseStudy 2: Question-Answering Agent with External Browser Integration The results are shown in Table 4. In this experiment, the setup is identical to Case Study 1. However, instead of embedding additional information directly into the prompt template, we perform a Wikipedia search to retrieve relevant answers, which are then inserted into the context for the question-answering process. For fairness, the retrieved information is simplified and limited to 500 characters. Compared to Case Study 1, we introduce the LLMCompiler framework, which allows parallel processing of tasks by decomposing them into independent components. For example, in the query “Are both Duke Energy and Affiliated

Table 4. The Accuracy and Latency Results of Case Study 2

LLM-SH	Accuracy	Latency
LangChain	61.80%	7.73s
Semantic Kernel	61.77%	7.85s
Haystack	61.75%	7.92s
Griptape	61.75%	7.78s
LLM-chain	61.80%	7.81s
LLMCompiler	62.00%	5.21s

Managers Group based in Massachusetts?”, external knowledge about Duke Energy and Affiliated Managers Group is required. Since there is no dependency between these two entities, LLMCompiler executes both external search tasks in parallel, improving efficiency. The experimental results, shown in the figure, demonstrate that LLMCompiler achieves a nearly 1.52× speedup compared to other prompt-based frameworks, significantly enhancing the performance for complex tasks. This improvement has substantial practical implications.

◇ **Practical Recommendations:** For complex question-answering tasks involving external browser functionality, the LLMCompiler framework is ideal. It enhances efficiency by enabling parallel task processing, particularly for queries requiring concurrent retrieval of multiple independent information sources.

6.2 Challenges Confronting the Prompting Framework

In this section, we will analyze the challenges and opportunities that the PF currently faces in terms of functionality implementation and issues related to safety and ethics.

6.2.1 Security Mechanisms of Prompting Framework. For a developed tool, both functionality and security mechanisms are equally crucial. Software that cannot guarantee the security of user information and is not harmless to society, regardless of its robust functionality, is unlikely to gain user support. Existing evidence suggests that LLMs and relatively mature PFs like LangChain have certain security issues [6, 69, 76, 101, 134]. However, the existing PFs have paid minimal attention to the significance of security ethics and privacy protection. Due to LLMs possessing an “Achilles’ heel” – being uncontrollable generative AI, we cannot predict the outputs, which can be fatal in certain scenarios. Therefore, we argue that the security mechanisms that PFs urgently need to enhance should consist of two parts: defense against prompt-based attacks and safeguarding the behavior of LLMs.

Defense Against Prompt-based Attacks. We introduce Prompt-based Attacks and suggest strategies to mitigate them in future PFs. While Prompt Engineering aims to optimize outputs through carefully crafted instructions, Prompt-based Attacks take a malicious approach, aiming to manipulate models via harmful inputs. These attacks, which can bypass model safeguards, vary in severity based on the model’s capabilities [31, 46, 69, 103, 140]. Prompt-based Attacks attempt to generate content contrary to the developer’s intent, typically through two methods: Prompt-based Deception (using linguistic techniques to bypass scrutiny) and Prompt-based Injection (tampering with instructions). Although these attacks might seem harmless in content generation contexts, they pose greater risks as PFs like AutoGPT grant more users execution authority over LLMs. Such attacks can lead to serious threats, such as creating e-mail worms, poisoning systems, or even manipulating files and funds. For defense, we recommend several instruction design strategies. First, use delimiters to separate instructions from content, ensuring user-generated content is clearly marked. This is a best practice endorsed by OpenAI and Andrew Ng. Second, place critical instructions at the end of prompts, as malicious instructions often ignore earlier ones. Lastly, implement a pre-filtering layer with whitelists and blacklists. Whitelists allow only specific inputs (e.g., Chinese characters for a Chinese-to-English translation model), while blacklists block harmful phrases or those violating regulations.

Safeguards of LLMs’ Behavior. LLMs sometimes exhibit undesirable behaviors, such as hallucinations or deception. To address this, adding a safeguard layer to their outputs is crucial. Enhancing the PF with such a mechanism ensures early interception and modification of objectionable content

before final results are generated. Microsoft's Nemo Guardrails, while helpful, has limitations. Its proprietary language may be user-friendly, but it lacks extensibility and turns model security into a manual policy configuration issue. This can lead to redundant configurations, limited flexibility, operational risks, and potentially reducing LLM capabilities. It appears to be a compromise solution, balancing LLM generation capabilities with the need for controllability and security. Future PFs should be built on mainstream programming languages, utilizing advanced technologies to automate and flexibly configure protection rules, focusing on three areas: topic relevance, content safety, and application security. Topic integrity safeguards ensure LLMs stay on track, preventing irrelevant outputs. Content safety safeguards address factual inaccuracies and harmful outputs, such as biased or offensive content. Application security restricts connections to trusted third-party applications, protecting LLMs from external attacks, like malware or network intrusion.

6.2.2 Capability Limitations of Prompting Framework. We analyze the capabilities of PFs across six key dimensions. While these frameworks perform well in most areas, there are notable limitations in implementation. Therefore, we identify three primary limitations: increasing learning curves, constraints in invoking external interfaces, and difficulties in handling multi-modal I/O.

Increasingly Steep Learning Curve. The rapid growth of LLMs has led to the proliferation of related tools and plugins, creating challenges in the development and maintenance of PFs. Software engineers must quickly integrate these new tools into existing frameworks, often resulting in functionality overlap and redundancy. While native features in PFs offer simple invocation via uniform interfaces, new additions often require users to call external packages with complex, non-uniform procedures, leading to a steeper learning curve. These complexities can also cause user experience issues and program bugs. Additionally, introducing new features may disrupt the functionality of existing ones. As noted, the technical documentation of PFs is often unclear. To address this, developers should focus on improving the framework's modularity, scalability, and standardization, ensuring future adaptability.

Constraints in Invoking External Interfaces. Current AI PFs excel in basic tasks like web browsing and database querying but struggle to integrate with specialized tools, including widely used applications like Microsoft Word, Excel, and PowerPoint. They also lack secure protocols for managing sensitive data in compliance with privacy regulations. These frameworks are mainly focused on text-based tasks, limiting their ability to process multimodal inputs, such as videos, Word documents, or e-mails, and restricting support for platforms like YouTube, arXiv, X, and Outlook. Future frameworks should expand support for both mainstream and emerging tools, as well as physical devices, to enhance the AI ecosystem's versatility.

7 Related Prompting Tools

In this section, we introduce practical tools that, while not part of the PF, complement it and facilitate the development of more complex and advanced LLM applications.

7.1 Efficient Deployment of LLMs.

Efficiently deploying LLMs locally and supporting application development represents a significant advancement. Notable frameworks include vLLM [57], Ollama [94], GPT4ALL [45], and llama.cpp [72]. vLLM optimizes memory and inference through PagedAttention and specialized components like BlockAllocator. Its LLMEngine manages tasks such as model loading, scheduling, and memory allocation. In GPU environments, vLLM outperforms Ollama, especially with concurrent and multi-GPU requests. Ollama, developed in Go, is a lightweight framework that supports local LLM execution with a user-friendly API. It excels in CPU-based inference and is ideal for

GPU-free environments. GPT4ALL is a Python SDK that simplifies LLM deployment and inference in conversational settings, offering an easy-to-use API for local use. llama.cpp, a highly optimized C/C++ implementation, focuses on efficient local LLM inference across different hardware using the GGML library for runtime support. These tools provide unique features to help developers efficiently deploy LLMs locally, accelerate inference, and enhance collaboration with PFs.

7.2 Evaluating Benchmark and Toolkit of LLMs.

Evaluating LLMs is crucial for understanding their capabilities and limitations, guiding human-machine interactions, and ensuring safety and reliability, as the choice of LLM directly impacts task performance. Evaluating LLMs has always been a crucial topic to ensure their reliability, safety, usability, and compliance while helping identify potential issues and improvement areas [5, 52, 65, 109]. Evals [95] is an open-source framework for LLM evaluation, providing easy-to-use benchmarks with minimal code. PromptBench [155] focuses on evaluating LLMs' robustness under adversarial prompts, offering infrastructure for simulating prompt attacks. PromptInject [103] modularly assembles prompts to assess LLM robustness against such attacks. Recently, S-Eval [147] advanced security evaluation with a multidimensional benchmark for LLMs. Its core feature is an automatic framework for test prompt generation, using expert LLMs to create high-quality test suites with baseline and attack prompts. S-Eval introduces a four-tier risk classification system across eight dimensions, helping users understand LLM strengths and weaknesses and select the most suitable models for content security.

8 Conclusion Future Directions

In this section, we summarize the key points discussed in this review and outline potential future directions based on the findings and the current state of the field.

8.1 Conclusion of Existing Prompting Frameworks

In this article, we explore the origins of the PF and its technological foundations, providing a conceptual definition and formal description. We classify PFs into three categories based on usage and technical attributes: the shell of LLMs (LLM-SH), language for interaction with LLMs (LLM-LANG), output restrictors of LLMs (LLM-RSTR). We conduct a comparative analysis of these frameworks to examine their compatibility, capabilities, documentation, and community support. We also highlight challenges in developing PFs and introduce relevant tools outside the framework domain that assist LLMs in task execution. Finally, we discuss the applicability and limitations of existing frameworks. While current PFs address the limitations of LLMs in real-world applications, challenges remain. These frameworks, with a different focus on documentation, compatibility, and external tool integration, offer compromise solutions for today's commercial needs. However, they lack robust security mechanisms and have inherent capability limitations, preventing them from being fully future-proof.

8.2 Future Directions

In advancing the capabilities of language models (LLMs), it is essential that the forthcoming generation of PFs overcome the limitations of current systems. By synthesizing the strengths of the three PFs discussed in this article, a transformative shift in user experience is achievable.

Enhanced Streamlining. The next iteration of the PF will focus on efficiency in two key areas. First, it will streamline user-LLM interactions, minimizing input effort. Second, it will optimize the interface between LLMs and their environments, covering both software and hardware. Technically, the framework will be designed for seamless compatibility with emerging products and technologies. Instructional materials will be tailored for both beginners and experts, ensuring a smooth learning

curve. The goal is to simplify complexity without compromising functionality, aligning with the principle that “simplicity is the ultimate sophistication.”

Fortified Security. Given the growing emphasis on data privacy and security, the next-generation PF will focus on strong safeguards. It will ensure that LLM-generated content complies with the highest security and privacy standards throughout the input, processing, and output stages. The framework will act as a dual shield, protecting users from malicious uses of LLMs while securing the LLMs and associated systems from external threats. By employing advanced encryption, access controls, and continuous monitoring, it will create a secure environment for all stakeholders.

Remarkable Versatility. The future of PFs depends on their ability to integrate with diverse external applications. Next-generation frameworks will enhance this by connecting effectively with various software across multiple domains. This flexibility will enable LLMs to make significant contributions in areas such as healthcare (e.g., diagnosis, treatment planning, patient education), research (e.g., accelerating discoveries, analyzing datasets, generating hypotheses), education (e.g., personalized tutoring, curriculum design, assessment), and transportation (e.g., optimizing traffic, predicting maintenance, improving logistics). This adaptability will drive innovation across industries.

Strict Standardization. To ensure interoperability and reliability, the next-generation PF will adhere to established domain-specific standards. These standards, critical for quality assurance across industries, include rules, guidelines, and best practices. For example, compliance with ISO 27001 for information security management will be a key requirement in IT, while healthcare will follow **Good Manufacturing Practice (GMP)** standards to ensure LLM-assisted processes are safe and effective. By aligning with these global benchmarks, the framework simplifies user experience and fosters collaboration, allowing diverse frameworks to coexist and strengthen the broader LLM ecosystem.

The Organic LLM Ecosystem. The evolution centers on an organic LLM ecosystem, where the next-generation PF acts as the core infrastructure, unifying all stages of the LLM lifecycle. It will enable developers to design, test, refine models, compare performance, and share insights. For end-users, the framework provides an intuitive interface for information retrieval, content generation, and problem-solving. This integrated feedback loop will drive technological progress and foster applications that are both more efficient and aligned with societal needs.

In conclusion, the next-generation PF represents a major step forward in LLM development, combining efficiency, security, versatility, and standardization into a unified system. By integrating model development, deployment, and user interaction, it has the potential to drive innovation across various sectors while ensuring system and user safety. As LLM capabilities evolve, this framework will serve as a foundation for creating intelligent, adaptable, and secure solutions to address the challenges of a rapidly changing technological landscape.

References

- [1] Fatih Kadir Akin. 2022. *Awesome ChatGPT Prompts*. Retrieved December 20, 2022 from <https://github.com/f/awesome-chatgpt-prompts>
- [2] Mostafa M. Amin, Erik Cambria, and Björn W. Schuller. 2023. Will affective computing emerge from foundation models and general artificial intelligence? A first evaluation of ChatGPT. *IEEE Intelligent Systems* 38, 2 (2023), 15–23.
- [3] AutoRAG. 2024. *AutoRAG*. Retrieved May 1, 2024 from <https://github.com/topics/open-source>
- [4] Stephen H. Bach, Victor Sanh, Zheng-Xin Yong, Albert Webson, Colin Raffel, Nihal V. Nayak, Abheesht Sharma, Taewoon Kim, M. Saiful Bari, Thibault Fevry, et al. 2022. PromptSource: An integrated development environment and repository for natural language prompts. (May 2022), 93–104. DOI: <https://doi.org/10.18653/v1/2022.acl-demo.9>
- [5] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. (Nov. 2023), 675–718. DOI: <https://doi.org/10.18653/v1/2023.ijcnlp-main.45>

- [6] Clark Barrett, Brad Boyd, Ellie Burzstein, Nicholas Carlini, Brad Chen, Jihye Choi, Amrita Roy Chowdhury, Mihai Christodorescu, Anupam Datta, Soheil Feizi, et al. 2023. Identifying and mitigating the security risks of generative AI. *Foundations and Trends in Privacy and Security* 6, 1 (12 2023), 1–52. DOI : <https://doi.org/10.1561/33000000041>
- [7] Yoshua Bengio, Aaron Courville, and Pascal Vincent. 2013. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, 8 (2013), 1798–1828.
- [8] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in Neural Information Processing Systems* 13 (2000). https://proceedings.neurips.cc/paper_files/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf
- [9] Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. 2023. Unlimiformer: Long-range transformers with unlimited length input. Article 1543 (2023), 22.
- [10] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1946–1969.
- [11] Rishi Bommasani, Percy Liang, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. 2023. Holistic Evaluation of Language Models. (2023). arXiv:2211.09110 [cs.CL] <https://arxiv.org/abs/2211.09110>
- [12] botpress. 2023. *Botpress*. Retrieved March 1, 2023 from <https://github.com/botpress/botpress>
- [13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33, Article 159 (2020), 25.
- [14] Jake Bruckman. 2023. *gpt-jargon*. Retrieved March 11, 2023 from <https://github.com/jbrukh/gpt-jargon>
- [15] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. 2023. Scaling transformer to 1M tokens and beyond with RMT. arXiv abs/2304.11062 (2023). <https://api.semanticscholar.org/CorpusID:258291566>
- [16] Yihan Cao, Siyu Li, Yixin Liu, Zhiling Yan, Yutong Dai, Philip Yu, and Lichao Sun. 2025. A Survey of AI-Generated Content (AIGC). *ACM Comput. Surv.* 57, 5, Article 125 (Jan. 2025), 38. DOI : <https://doi.org/10.1145/3704262>
- [17] Yong Cao, Li Zhou, Seolhwa Lee, Laura Cabello, Min Chen, and Daniel Hershcovich. 2023. Assessing cross-cultural alignment between chatgpt and human societies: An empirical study. (May 2023), 53–67. DOI : <https://doi.org/10.18653/v1/2023.c3nlp-1.7>
- [18] Marco Cascella, Jonathan Montomoli, Valentina Bellini, and Elena Bignami. 2023. Evaluating the feasibility of ChatGPT in healthcare: An analysis of multiple clinical and research scenarios. *Journal of Medical Systems* 47, 1 (2023), 33.
- [19] Chainlit. 2024. *Chainlit*. Retrieved January 30, 2024 from <https://docs.chainlit.io/get-started/overview>
- [20] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.* 15, 3, Article 39 (March 2024), 45. DOI : <https://doi.org/10.1145/3641289>
- [21] Harrison Chase. 2022. *LangChain*. Retrieved December 10, 2022 from <https://github.com/hwchase17/langchain>
- [22] Le Chen, Pei-Hung Lin, Tristan Vanderbruggen, Chunhua Liao, Murali Emani, and Bronis de Supinski. 2023. LM4HPC: Towards effective language model application in high-performance computing. (2023), 18–33. DOI : https://doi.org/10.1007/978-3-031-40744-4_2
- [23] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=cSimKw5p6R>
- [24] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. 2023. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. <https://openreview.net/forum?id=EHg5GDnyq1>
- [25] Xuanting Chen, Junjie Ye, Can Zu, Nuo Xu, Rui Zheng, Minlong Peng, Jie Zhou, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. How robust is GPT-3.5 to predecessors? A comprehensive study on language understanding tasks. arXiv abs/2303.00293 (2023). <https://api.semanticscholar.org/CorpusID:257255121>
- [26] Ajay Kumar Chintala and Vignesh Aigal. [n. d.]. *LLMStack: A Platform to Build and Deploy LLM Applications*. Retrieved January 30, 2024 from <https://github.com/trypromptly/llmstack>
- [27] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. PaLM: Scaling language modeling with pathways. (2023), 113.

- [28] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *J. Mach. Learn. Res.* 25, 1, Article 70 (Jan. 2024), 53.
- [29] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel Kuksa. 2011. Natural language processing (almost) from scratch. *Journal of Machine Learning Research* 12, ARTICLE (2011), 2493–2537.
- [30] Wei Dai, Jionghao Lin, Flora Jin, Tongguang Li, Yi-Shan Tsai, Dragan Gasevic, and Guanliang Chen. 2023. Can large language models provide feedback to students? A case study on ChatGPT. (2023), 323–325. DOI: <https://doi.org/10.1109/ICALT58122.2023.00100>
- [31] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2024. Jailbreaker: Automated jailbreak across multiple large language model chatbots. (2024). DOI: <https://doi.org/10.14722/ndss.2024.24188>
- [32] Ameet Deshpande, Vishvak Murahari, Tanmay Rajpurohit, Ashwin Kalyan, and Karthik Narasimhan. 2023. Toxicity in ChatGPT: Analyzing persona-assigned language models. (Dec. 2023), 1236–1270. DOI: <https://doi.org/10.18653/v1/2023.findings-emnlp.88>
- [33] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. (2019). <https://api.semanticscholar.org/CorpusID:52967399>
- [34] Dify. 2024. *Dify*. Retrieved January 30, 2024 from <https://dify.ai/>
- [35] Etienne Dlocker, Bob van Luijt, Byron Voorbach, Mohd Shukri Hasan, Abdel Rodriguez, Dirk Alexander Kulawiak, Marcin Antas, and Parker Duckworth. [n. d.]. *Weaviate*. Retrieved January 30, 2024 from <https://github.com/weaviate/weaviate>
- [36] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. 2024. A survey for in-context learning. (Nov. 2024), 1107–1128. DOI: <https://doi.org/10.18653/v1/2024.emnlp-main.64>
- [37] Hongyang Du, Zonghang Li, Dusit Niyato, Jiawen Kang, Zehui Xiong, Xuemin Shen, and Dong In Kim. 2024. Enabling AI-generated content (AIGC) services in wireless edge networks. *IEEE Wireless Communications* 31, 3 (2024), 226–234. DOI: <https://doi.org/10.1109/MWC.004.2300015>
- [38] Yogesh K. Dwivedi, Nir Kshetri, Laurie Hughes, Emma Louise Slade, Anand Jeyaraj, Arpan Kumar Kar, Abdullah M. Baabdullah, Alex Koochang, Vishnupriya Raghavan, Manju Ahuja, et al. 2023. “So what if ChatGPT wrote it?” Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. *International Journal of Information Management* 71, C (Aug. 2023), 63. DOI: <https://doi.org/10.1016/j.ijinfomgt.2023.102642>
- [39] Eric Elliott. 2023. *SudoLang*. Retrieved January 30, 2023 from <https://github.com/paralleldrive/sudolang-llm-support>
- [40] Emilio Ferrara. 2023. Should chatgpt be biased? Challenges and risks of bias in large language models. *First Monday* 28, 11 (Nov. 2023). DOI: <https://doi.org/10.5210/fm.v28i11.13346>
- [41] Flappy. 2024. *Flappy*. Retrieved January 30, 2024 from <https://flappy.pleisto.com/>
- [42] Floom. 2024. *Floom*. Retrieved January 30, 2024 from <https://floomai.github.io/>
- [43] Forethought-Technologies. 2023. *AutoChain*. Retrieved March 10, 2023 from <https://github.com/Forethought-Technologies/AutoChain>
- [44] gpt-prompt engineer. 2024. *gpt-Prompt-Engineer*. Retrieved January 30, 2024 from <https://github.com/mshumer/gpt-prompt-engineer>
- [45] GPT4All. 2023. *GPT4All*. Retrieved June 10, 2023 from <https://www.nomic.ai/gpt4all>
- [46] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. More than you’ve asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models. arXiv abs/2302.12173 (2023). <https://api.semanticscholar.org/CorpusID:257102404>
- [47] griptape ai. 2023. *Griptape*. Retrieved March 1, 2023 from <https://github.com/griptape-ai/griptape>
- [48] henomis. 2023. *LinGoose*. Retrieved March 1, 2023 from <https://github.com/henomis/lingoose>
- [49] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, zili wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta programming for multi-agent collaborative framework. 23247–23275. https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf
- [50] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. arXiv abs/2106.09685 (2021). <https://api.semanticscholar.org/CorpusID:235458009>
- [51] InsuranceToolkits. 2023. *PromptFlow*. Retrieved March 1, 2023 from <https://github.com/InsuranceToolkits/promptflow>
- [52] Neel Jain, Khalid Saifullah, Yuxin Wen, John Kirchenbauer, Manli Shu, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. 2024. Bring your own data! Self-supervised evaluation for large language models. (2024). <https://openreview.net/forum?id=zH6zBoktYO>

- [53] Frederick Jelinek. 1998. *Statistical Methods for Speech Recognition*. MIT Press.
- [54] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael Mahoney, Kurt Keutzer, and Amir Gholami. 2023. An LLM compiler for parallel function calling. In *Proceedings of the International Conference on Machine Learning (ICML) 2025*.
- [55] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems* 35, Article 1613 (2022), 15.
- [56] Sotiris B. Kotsiantis, Ioannis Zaharakis, P. Pintelas, et al. 2007. Supervised machine learning: A review of classification techniques. *Emerging Artificial Intelligence Applications in Computer Engineering* 160, 1 (2007), 3–24.
- [57] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [58] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. 2001. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. (2001), 282–289.
- [59] Langroid. 2024. *Langroid*. Retrieved January 30, 2024 from <https://langroid.github.io/langroid/>
- [60] LangStream. 2024. *LangStream*. Retrieved January 30, 2024 from <https://langstream.ai/>
- [61] Md Tahmid Rahman Laskar, M. Saiful Bari, Mizanur Rahman, Md Amran Hossen Bhuiyan, Shafiq Joty, and Jimmy Xi-angji Huang. 2023. A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets. (July 2023), 431–469. DOI : <https://doi.org/10.18653/v1/2023.findings-acl.29>
- [62] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. (July 2020), 7871–7880. DOI : <https://doi.org/10.18653/v1/2020.acl-main.703>
- [63] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. 2022. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.). 35 (2022), 3843–3857. <https://openreview.net/forum?id=IFXTZERXdm7>
- [64] Xinzhe Li, Ming Liu, Shang Gao, and Wray Buntine. 2023. A survey on out-of-distribution evaluation of neural nlp models. Article 749 (2023), 9. DOI : <https://doi.org/10.24963/ijcai.2023/749>
- [65] Yen-Ting Lin and Yun-Nung Chen. 2023. LLM-Eval: Unified multi-dimensional automatic evaluation for open-domain conversations with large language models. *CoRR* abs/2305.13711 (2023). <https://doi.org/10.48550/arXiv.2305.13711>
- [66] Jerry Liu. 2022. *LlamaIndex*. DOI : <https://doi.org/10.5281/zenodo.1234>. Accessed: Dec. 10, 2022.
- [67] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [68] Siru Liu, Aileen P. Wright, Barron L. Patterson, Jonathan P. Wanderer, Robert W. Turer, Scott D. Nelson, Allison B. McCoy, Dean F. Sittig, and Adam Wright. 2023. Using AI-generated suggestions from ChatGPT to optimize clinical decision support. *Journal of the American Medical Informatics Association* 30, 7 (2023), 1237–1245.
- [69] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2025. Prompt injection attack against LLM-integrated applications. arXiv:2306.05499. Retrieved from <https://arxiv.org/abs/2306.05499>
- [70] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized bert pretraining approach. arXiv:1907.11692. Retrieved from <https://arxiv.org/abs/1907.11692>
- [71] Swiss Army Llama. 2024. *Swiss Army Llama*. Retrieved January 30, 2024 from https://github.com/Dicklesworthstone/swiss_army_llama
- [72] llama.cpp. 2023. *llama.cpp*. Retrieved March 10, 2023 from <https://github.com/ggerganov/llama.cpp>
- [73] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. Article 1882 (2023), 32.
- [74] Brady D. Lund, Ting Wang, Nishith Reddy Mannuru, Bing Nie, Somipam Shimray, and Ziang Wang. 2023. ChatGPT and a new academic reality: Artificial intelligence-written research papers and the ethics of the large language models in scholarly publishing. *Journal of the Association for Information Science and Technology* 74, 5 (2023), 570–581.
- [75] Magentic. 2024. *Magentic*. Retrieved January 30, 2024 from <https://magentic.dev/>
- [76] Kai Mei, Zheng Li, Zhenting Wang, Yang Zhang, and Shiqing Ma. 2023. NOTABLE: Transferable backdoor attacks against prompt-based NLP models. (July 2023), 15551–15565. DOI : <https://doi.org/10.18653/v1/2023.acl-long.867>
- [77] MemFree. 2024. *MemFree*. Retrieved January 30, 2024 from <https://github.com/memfreemem/memfree>
- [78] David Mezzetti. 2020. *txtai: the all-in-one embeddings database*. Retrieved June 10, 2020 from <https://github.com/neuml/txtai>

- [79] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. Augmented language models: A survey. arXiv:2302.07842. Retrieved from <https://arxiv.org/abs/2302.07842>
- [80] microsoft. 2023. *Guidance*. Retrieved December 30, 2023 from <https://github.com/guidance-ai/guidance>
- [81] microsoft. 2023. *TypeChat*. Retrieved December 30, 2023 from <https://github.com/microsoft/TypeChat>
- [82] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:5959482>
- [83] Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernocký, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *Interspeech*, Vol. 2. Makuhari, 1045–1048.
- [84] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* 26 (2013), 3111–3119.
- [85] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Comput. Surv.* 56, 2, Article 30 (Sep 2023), 40 pages. DOI : <https://doi.org/10.1145/3605943>
- [86] Sewon Min, Xinxin Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the role of demonstrations: What makes in-context learning work? (Dec. 2022), 11048–11064. DOI : <https://doi.org/10.18653/v1/2022.emnlpmain.759>
- [87] MindSQL. 2024. *MindSQL*. Retrieved January 30, 2024 from <https://github.com/MindInventory/MindSQL>
- [88] ModelFusion. 2024. *ModelFusion*. Retrieved from <https://modelfusion.dev/guide/>
- [89] Marta Montenegro-Rueda, José Fernández-Cerero, José María Fernández-Batanero, and Eloy López-Meneses. 2023. Impact of the implementation of chatgpt in education: A systematic review. *Computers* 12, 8 (2023), 153.
- [90] Yohei Nakajima. 2023. *BabyAGI*. Retrieved January 30, 2024 from <https://github.com/yoheinakajima/babyagi>
- [91] OpenAI. 2024. Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [92] NVIDIA. 2023. *NeMo-Guardrails*. Retrieved January 30, 2024 from <https://github.com/NVIDIA/NeMo-Guardrails>
- [93] Franz Josef Och, Daniel Gildea, Sanjeev Khudanpur, Anoop Sarkar, Kenji Yamada, Alexander Fraser, Shankar Kumar, Libin Shen, David A. Smith, Katherine Eng, et al. 2004. A smorgasbord of features for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*. 161–168.
- [94] Ollama. 2023. *Ollama*. Retrieved January 30, 2024 from <https://ollama.com/>
- [95] OpenAI. 2023. *Evals*. Retrieved January 30, 2024 from <https://github.com/openai/evals>
- [96] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]. Retrieved from <https://arxiv.org/abs/2303.08774>
- [97] OpenDAN.ai. 2023. *OpenDAN*. Retrieved January 30, 2024 from <https://www.opendan.ai/>
- [98] Jonas Oppenlaender and Joonas Hämäläinen. 2023. Mapping the challenges of HCI: An application and evaluation of ChatGPT and GPT-4 for cost-efficient question answering.
- [99] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35, Article 2011 (2022), 15.
- [100] Ankit Pal. 2022. Promptify: Structured output from LLMs. Retrieved from <https://github.com/prompts-lab/Promptify>. Prompt-Engineering components for NLP tasks in Python.
- [101] Rodrigo Pedro, Daniel Castro, Paulo Carreira, and Nuno Santos. 2023. From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application? arXiv:2308.01990. Retrieved from <https://arxiv.org/abs/2308.01990>
- [102] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction tuning with gpt-4. arXiv:2304.03277. Retrieved from <https://arxiv.org/abs/2304.03277>
- [103] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. arXiv:2211.09527. Retrieved from <https://arxiv.org/abs/2211.09527>
- [104] Malte Pietsch, Timo Möller, Bogdan Kostic, Julian Risch, Massimiliano Pippi, Mayank Jobanputra, Sara Zanzottera, Silvano Cerza, Vladimir Blagojevic, Thomas Stadelmann, et al. 2019. *Haystack: The End-to-End NLP Framework for Pragmatic Builders*. Retrieved January 30, 2024 from <https://github.com/deepset-ai/haystack>
- [105] Pinecone. 2023. *Pinecone*. Retrieved January 30, 2024 from <https://github.com/pinecone-io>
- [106] PromptHub. 2023. *PromptHub*. Retrieved January 30, 2024 from <https://github.com/deepset-ai/prompthub>
- [107] QA-Pilot. 2024. *QA-Pilot*. Retrieved January 30, 2024 from <https://github.com/reid41/QA-Pilot>
- [108] QAnything. 2024. *QAnything*. Retrieved January 30, 2024 from <https://qanything.ai/>
- [109] Chengwei Qin, Aston Zhang, Zhuosheng Zhang, Jiaao Chen, Michihiro Yasunaga, and Diyi Yang. 2023. Is ChatGPT a general-purpose natural language processing task solver? In *The 2023 Conference on Empirical Methods in Natural Language Processing*. <https://openreview.net/forum?id=u03xn1COsO>

- [110] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi, Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong, Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan, Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng Yang, Tongshuang Wu, Heng Ji, Guoliang Li, Zhiyuan Liu, and Maosong Sun. 2024. Tool learning with foundation models. *ACM Comput. Surv.* 57, 4, Article 101 (Dec. 2024), 40. DOI: <https://doi.org/10.1145/3704435>
- [111] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. (2019).
- [112] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
- [113] Matt Rickard. 2023. *ReLLM*. Retrieved January 30, 2024 from <https://github.com/r2d4/rellm>
- [114] Ronald Rosenfeld. 2000. Two decades of statistical language modeling: Where do we go from here? *Proc. IEEE* 88, 8 (2000), 1270–1278.
- [115] ruvnet. 2023. *PromptLang*. Retrieved January 30, 2024 from <https://github.com/ruvnet/promptlang>
- [116] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. 2021. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research* 304 (2021), 114135. DOI: <https://doi.org/10.1016/j.psychres.2021.114135>
- [117] Sentdex. 2023. *TermGPT*. Retrieved January 30, 2024 from <https://github.com/Sentdex/TermGPT>
- [118] ShreyaR. 2023. *Guardrails*. Retrieved January 30, 2024 from <https://github.com/ShreyaR/guardrails>
- [119] Significant Gravitas. [n. d.]. *Auto-GPT*. Retrieved January 30, 2024 from <https://github.com/Significant-Gravitas/Auto-GPT>
- [120] Taranjeet Singh. 2023. *Embedchain*. Retrieved January 30, 2024 from <https://github.com/embedchain/embedchain>
- [121] sobelio. 2023. *llm-chain*. Retrieved January 30, 2024 from <https://github.com/sobelio/llm-chain>
- [122] Scott M. Thede and Mary Harper. 1999. A second-order hidden Markov model for part-of-speech tagging. In *Proceedings of the 37th Annual Meeting of the Association for Computational Linguistics*. 175–182.
- [123] TimPietrusky. 2023. *Hyv*. Retrieved January 30, 2024 from <https://github.com/failfa-st/hyv>
- [124] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from <https://arxiv.org/abs/2302.13971>
- [125] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from <https://arxiv.org/abs/2307.09288>
- [126] TransformerOptimus. 2023. *SuperAGI*. Retrieved January 30, 2024 from <https://github.com/TransformerOptimus/SuperAGI>
- [127] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017), 6000–6010.
- [128] Gil LaHaye Vic Perdana. 2023. *Semantic-kernel*. Retrieved March 1, 2023 from <https://github.com/microsoft/semantic-kernel>
- [129] Jindong Wang, Xixu Hu, Wenxin Hou, Hao Chen, Runkai Zheng, Yidong Wang, Linyi Yang, Haojun Huang, Wei Ye, Xiubo Geng, et al. 2023. On the robustness of chatgpt: An adversarial and out-of-distribution perspective. In *ICLR 2023 Workshop on Trustworthy and Reliable Large-Scale Machine Learning Models*. <https://openreview.net/forum?id=uw6H5kgoM29>
- [130] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [131] Xinlong Wang, Wen Wang, Yue Cao, Chunhua Shen, and Tiejun Huang. 2023. Images speak in images: A generalist painter for in-context visual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6830–6839.
- [132] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-instruct: Aligning language model with self generated instructions. (July 2023), 13484–13508. DOI: <https://doi.org/10.18653/v1/2023.acl-long.754>
- [133] Taylor Webb, Keith J. Holyoak, and Hongjing Lu. 2023. Emergent analogical reasoning in large language models. *Nature Human Behaviour* 7, 9 (2023), 1526–1541. DOI: <https://doi.org/10.1038/s41562-023-01659-w>
- [134] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How does LLM safety training fail? 36 (2023), 80079–80110. https://proceedings.neurips.cc/paper_files/paper/2023/file/fd6613131889a4b656206c50a8bd7790-Paper-Conference.pdf

- [135] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *Transactions on Machine Learning Research* (2022). <https://openreview.net/forum?id=yzkSU5zdwD>
- [136] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35, Article 1800 (2022), 14.
- [137] Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, et al. 2023. Larger language models do in-context learning differently. arXiv:2303.03846. Retrieved from <https://arxiv.org/abs/2303.03846>
- [138] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Clara Ma, Yacine Jernite, Julien Plu, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Qun Liu and David Schlangen (Eds.). Association for Computational Linguistics, 38–45. DOI : <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [139] Changrong Xiao, Sean Xin Xu, Kunpeng Zhang, Yufang Wang, and Lei Xia. 2023. Evaluating reading comprehension exercises generated by LLMs: A showcase of chatgpt in education applications. In *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)*. 610–625.
- [140] Jun Yan, Vikas Yadav, Shiyang Li, Lichang Chen, Zheng Tang, Hai Wang, Vijay Srinivasan, Xiang Ren, and Hongxia Jin. 2024. Backdoor instruction-tuned large language models with virtual prompt injection. <https://openreview.net/forum?id=b3KgHQos7P>
- [141] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Bing Yin, and Xia Hu. 2024. Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond. *ACM Trans. Knowl. Discov. Data* 18, 6, Article 160 (April 2024), 32. DOI : <https://doi.org/10.1145/3649506>
- [142] Xianjun Yang, Yan Li, Xinlu Zhang, Haifeng Chen, and Wei Cheng. 2023. Exploring the limits of chatgpt for query or aspect-based text summarization. arXiv:2302.08081. Retrieved from <https://arxiv.org/abs/2302.08081>
- [143] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R. Salakhutdinov, and Quoc V. Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in Neural Information Processing Systems* 32, Article 517 (2019), 11.
- [144] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. (Oct.–Nov. 2018), 2369–2380. <https://doi.org/10.18653/v1/D18-1259>
- [145] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=5Xc1ecxO1h>
- [146] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. arXiv:2210.03629. Retrieved from <https://arxiv.org/abs/2210.03629>
- [147] Xiaohan Yuan, Jinfeng Li, Dongxia Wang, Yuefeng Chen, Xiaofeng Mao, Longtao Huang, Hui Xue, Wenhui Wang, Kui Ren, and Jingyi Wang. 2025. S-Eval: Automatic and adaptive test generation for benchmarking safety evaluation of large language models. In *Proceedings of the ACM Softw. Eng.* 2, ISSTA, Article ISSTA094 (June 2025), 22. DOI : <https://doi.org/10.1145/3728971>
- [148] Zhanpeng Zeng, Cole Hawkins, Mingyi Hong, Aston Zhang, Nikolaos Pappas, Vikas Singh, and Shuai Zheng. 2023. VCC: Scaling transformers to 128K tokens or more by prioritizing important tokens. Article 889 (2023), 27.
- [149] Chaoning Zhang, Chenshuang Zhang, Chenghao Li, Yu Qiao, Sheng Zheng, Sumit Kumar Dam, Mengchun Zhang, Jung Uk Kim, Seong Tae Kim, Jinwoo Choi, et al. 2023. One small step for generative AI, one giant leap for AGI: A complete survey on ChatGPT in AIGC era. arXiv:2304.06488. Retrieved from <https://arxiv.org/abs/2304.06488>
- [150] Chaoning Zhang, Chenshuang Zhang, Sheng Zheng, Yu Qiao, Chenghao Li, Mengchun Zhang, Sumit Kumar Dam, Chu Myaet Thwal, Ye Lin Tun, Le Luang Huy, et al. 2023. A complete survey on generative AI (AIGC): Is ChatGPT from GPT-4 to GPT-5 all you need? arXiv:2303.11717. Retrieved from <https://arxiv.org/abs/2303.11717>
- [151] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2026. A survey of large language models. arXiv:2303.18223. Retrieved from <https://arxiv.org/abs/2303.18223>
- [152] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody_Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. Efficiently programming large language models using sglang. *Article* 2000 (2024), 27.
- [153] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-most prompting enables complex reasoning in large

- language models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=WZH7099tgfM>
- [154] Jiaming Zhou, Tengyue Li, Simon James Fong, Nilanjan Dey, and Rubén González Crespo. 2023. Exploring chatGPT'S potential for consultation, recommendations and report diagnosis: Gastric cancer and gastroscopy reports' case. *International Journal of Interactive Multimedia & Artificial Intelligence* 8, 2 (2023), 7–13.
 - [155] Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Gong, and Xing Xie. 2024. PromptBench: Towards evaluating the robustness of large language models on adversarial prompts. (2024), 57–68. DOI : <https://doi.org/10.1145/3689217.3690621>

Appendix

A Popular Prompting Datasets

Table 5. The Collection of the Popular Prompt Datasets

Dataset	#Scale	What is it – Content	How to use it – Example
Awesome-ChatGPT-Prompts	171	Awesome-ChatGPT-Prompts contains prompts for various topics/roles, which are system prompts for specific scenarios, allowing LLMs to mimic a Linux terminal, JavaScript console, Excel page, and more	Directly choose a prompt according to the scenario required. An example is: Role: Python interpreter Prompt: <i>I want you to act like a Python interpreter. I will give you Python code, and you will execute it. Do not provide any explanations. Do not respond with anything except the output of the code. The first code is: p“rint('hello world!')”</i>
Prompt Source	2000	PromptSource contains diverse prompt templates to be filled	PromptSource provides a Web-based GUI that enables developers to write prompts in a template language and immediately view their outputs on different examples
GPTs	272	GPTs is a dataset that collects leaked prompts of LLMs	GPTs gives a debugged ChatGPT interface/GUI, and specific scenario communication can be carried out in this scenario without entering additional prompts
BuzzVault	500	BuzzVault is a carefully crafted prompt set specifically for marketers and related scenarios	Directly choose a prompt for marketers. An example is: Prompt: <i>I’m looking to create visually appealing social media posts for {social media platform} to catch the attention of potential customers. What design principles should I keep in mind, and what tools can I use to enhance my posts, such as {tool 1}, {tool 2}, and {tool 3}?</i>
Prompt-Eng Dataset	1000	Prompt-Eng Dataset offers 1,000 prompts in 10 categories: Open Ended QA, Classification, Roleplaying, Data manipulation, Language Translation, Problem Solving, Logical Reasoning, Coding, Writing, Text Summarization	Directly choose a prompt according to the category required. An example is: Prompt: <i>I have to come up with ideas for California walnuts for my marketing agency - like social media ideas and influencer collabs. Instead of the regular recipe collabs or health benefits collabs, I want cool, quirky, out of the box ideas.</i>
YiVal	19	YiVal is a tool to streamline the tuning process for prompts and any configurations in the loop. YiVal means Your Automatic Prompt Engineering Assistant	Create a configuration file(.yaml) for automated test dataset generation from a template demo. An example is: Demo: Build a Customized Travel Guide Bot Prompt: <i>Leverage automated prompts inspired by the travel community’s most popular suggestions, such as those from awesome-chatgpt-prompts.</i>
Gpt-Prompt-Engineer	Generator	Gpt-Prompt-Engineer allows just simply input a description of your task and some test cases, and the system will generate, test, and rank a multitude of prompts to find the ones that perform the best.	Directly input the form, content, and quantity of the desired prompts, and the system will provide the prompts, score them, and sort each generated prompt accordingly.
PromptHub	Generator	PromptHub generates prompt templates optimized for the chosen model in just a few clicks, with best practices built-in.	Directly enter the request and the desired scenario to enable the system to generate optimized prompts based on its best practices.
Promptify	Generator	Promptify is a Python library dedicated to natural language processing (NLP) tasks, utilizing GPT or other LLMs to generate structured output.	Enter the query and the desired scenario directly, as well as specify the series of models to utilize (optional), such as GPT, PaLM, and others, to enable the system to generate different NLP Task prompts.

Received 18 November 2023; revised 3 January 2025; accepted 26 December 2025